UNIVERSITY of INFORMATION TECHNOLOGY and MANAGEMENT
in Rzeszow, POLAND

FACULTY OF INFORMATION TECHNOLOGY

Field of Study: INFORMATION TECHNOLOGY
Specialty: Programming

Pashazade Yusif

Student ID number: 70926

EasyAI: a Hybrid AI-Powered Educational Platform

Supervisor: Dr. inż. Łukasz Piątek

DIPLOMA THESIS AT FIRST-CYCLE STUDIES

Rzeszow 2026

Table of Contents

Introduction 3

State of the Art 4

Introduction 4

Comparative Analysis 4

Summary 6

1. Chapter 1 - Overview of Web Application 8

1.1 Web Applications 8

1.2 Frontend Technologies 8

1.3 Client-Server Architecture 9

1.4 HTTP Communication 10

1.5 HTML 12

1.6 CSS 13

1.7 JavaScript 14

2. Chapter 2 - Selected Tech Stack(s) 15

2.1 Python vs JavaScript 15

2.2 FastAPI Framework 16

2.3 Open-access LLM - Ollama 17

2.4 Database 19

3. Chapter 3 – Preliminary specification – Functional and Non-Functional Requirements 20

3.1 Functional Requirements 20

3.2 Non-functional Requirements 21

3.3 UML 22

4. Chapter 4 – Backend Architecture and Implementation 25

4.1 Layered architecture overview 25

4.2 API layer 27

4.3 Data access layer 28

5. Chapter 5-Presentation for End User and Admin role. 29
5.1 Normal user 29
5.1.1 The landing page 29
5.1.2 The Authentication page 30
5.1.3 Learning Level Section Page 31
5.1.4 Lesson Interface 32
5.1.5 AI Chatbot Assistant 33
5.1.6 Quiz Interface 34
5.2 Admin role 34
5.2.1 Admin panel 35

Summary 41
Bibliography 42
List of Figures 43

Introduction

Nowadays, it is obvious that AI is seeing significant demand across most industries. In general, users have different ways to communicate with GenAI. For example, instead of writing long texts, people prefer to talk with GenAI. ChatGPT is one of the most well-known examples that immediately comes to mind. Moreover, GenAI can be considered a great advantage for students. The number of people who rely on artificial intelligence in their daily lives to get knowledge is quite significant. Therefore, the main goal of this thesis is to create a modern and friendly design that helps students gain knowledge with visual and responsive interactive components. Obviously, anyone can effortlessly obtain tons of text-based materials via the internet. However, visual representation of information can facilitate comprehension more effectively than text-based materials. Moreover, most students prefer to use screenshots to describe problems with AI models rather than writing long and detailed texts. Then, instead of creating a simple AI chatbot, I decided to build an application that supports both text-based and multimedia explanations. Furthermore, the platform is planned to build a modern AI-powered system through hybrid integration of cloud-based and local language models.

Users can watch YouTube videos with theory-oriented materials. To enhance the adaptivity of the learning system, the quiz component was one of the best choices, which will be integrated into the application. Through a feedback-driven approach, users can improve their performance in understanding topics across multiple subjects. The system will also cover subjects such as mathematics, physics, and chemistry, which are widely considered challenging yet fundamental disciplines for students. Despite being challenging, the goal is not to develop an application to put some pressure on users to complete quizzes successfully. Each subject has a different number of topics that are most popular among students. Therefore, the goal is to develop an application that incorporates the features mentioned above.

This thesis is organized into 5 Chapters. Before 5 chapters, State of the Art will present an overview of other applications and their features, with a comparison to the EasyAI application through screenshots. The first chapter will present an overview of web apps. This chapter will state the fundamental concept of web application, including client-server architecture, web browser HTTP requests and responses, and routing mechanisms. Additionally, the roles of HTML, CSS, and JavaScript will be described in this chapter. Chapter 2 will explain technologies that will be used in applications, such as frameworks, programming languages, and open-access large language models (LLMs). This includes the Python programming language with the FastAPI framework for backend development and Ollama as an open-access LLM. Chapter 3 will state the preliminary specification of the application, which includes functional and non-functional requirements and UML diagrams. Additionally, it can be observed that both functional and non-functional requirements make the system more accessible to users. Chapter 4 will introduce the internal

working logic of the system, which covers authentication, backend logic, API logic, and so on. There are lots of functions in this chapter that improve system efficiency and performance. Chapter 5 will describe how the user interacts with the system and demonstrate its working state through screenshots. Furthermore, different user interfaces will be provided for both admin and non-registered users. Summary will state what will be done, how purpose will be achieved, and the results.

State of the Art

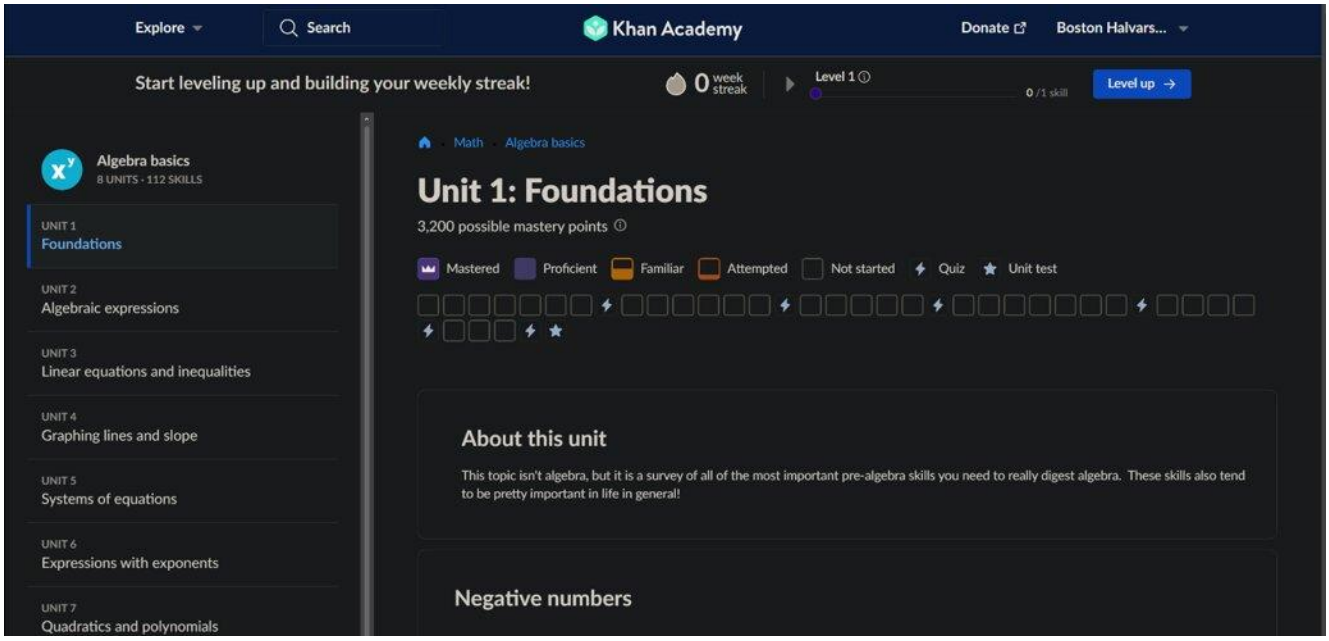
Introduction

In 2008, Khan Academy was created by Salman Khan, whose main purpose was to create a platform where everyone can learn educational subjects such as math, biology, physics, etc. The idea was simple and inspiring; however, implementing the mastery training concept was quite challenging. One of the prominent approaches to achieving this is though visual materials, just as Mr. Salman himself developed more than a thousand videos when he first initiated the project. While developing the application without integrating additional features, the system would not differ from standard educational applications. Mr. Salmon's idea of making education accessible to everyone inspired the goal of the thesis to develop an application with various functionalities and a user-friendly interface. An interface where you can find all the features you need. On the other hand, freeCodeCamp is an open-access educational platform focusing on programming, web development, and software development training. Learning through infrastructure is accessible on social media or web. However, the platform does not offer an accessible IDE on its web site. By adding an essential IDE could create user-friendly environment. Therefore, more of this will be discussed in other sections of State of the Art.

Comparative Analysis

The gaps in existing systems are identified personally. Both applications have their own gaps, which will be discussed separately. The proposed system aims to address the identified gaps in existing solutions. On Khan Academy, the course page displaying learning units (see on Fig.1) is clear and simple:

Fig. 1. Course page displaying study units

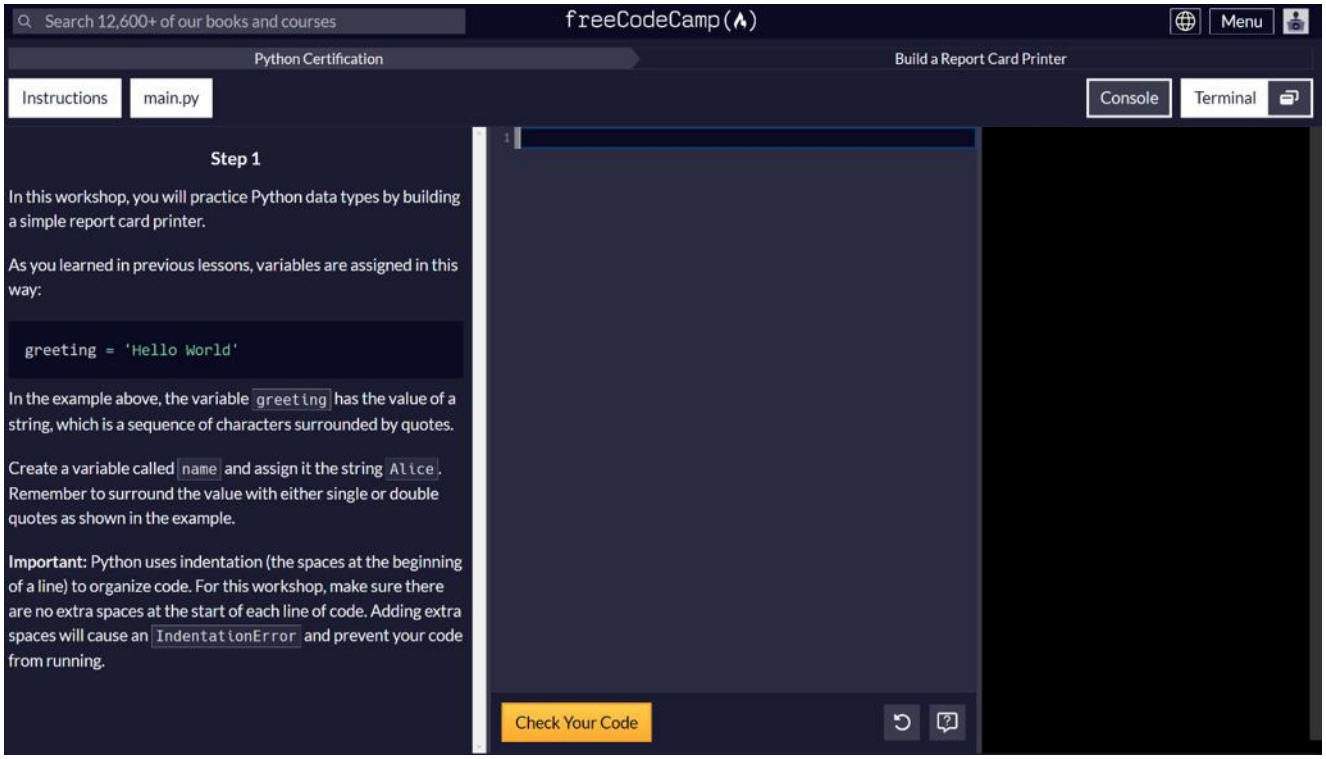


Source: https://www.khanacademy.org/math/algebra-basics/basic-alg-foundations, dated 10.05.2026

However, users cannot explicitly select their knowledge level, such as intermediate or expert. Since the platform provides structured learning paths rather than allowing users to choose their proficiency, it attracts beginner learners. Moreover, the application suggests watching a video relevant to the selected topic. Having visual material results in a noticeable improvement in the user's understanding of the topic. Also, Khan Academy covers a broad range of educational content, including mathematics, sciences, and computer science. Besides providing this feature, it supports the structure of a user-friendly environment. These gaps will be features on the application, which will affect the efficiency of the instructional path. Now another gap in freeCodeCamp will be discussed.

So, freeCodeCamp delivers a task-based study approach rather than offering a visual instructional approach like Khan Academy (see on Fig. 2). The system checks the user's response and marks the task as completed. To get a programming language certificate, users should finish all those tasks. Instead of offering only task-based progression, theoretical and informative learning paths are compulsory in applications:

Fig. 2. Python task with online compiler on FreeCodeCamp web application



Source: https://www.freecodecamp.org/learn/python-v9/workshop-report-card-printer/step-1, dated 11.05.2026

Like Khan Academy, both YouTube videos and website content target a broader audience. A key difference is that Khan Academy provides all content together on the web; however, freeCodeCamp supports separate learning content, which suggests freedom for users. For example, at Khan Academy, learners who struggle with written explanations can benefit from video material on the same page. Whereas another platform does not support this functionality. Unless users can benefit from recorded resources, they may struggle to understand the topic. Unlike Khan Academy, freeCodeCamp delivers specific training solutions that are related to programming and software development. Despite receiving funding from community and developer donations, the platform is widely used by millions to enhance software skills.

Summary

After analyzing and comparing existing platforms, several weaknesses were identified, including a lack of practical, user-friendly interfaces. The following system aims to minimize these issues during application design. The discussed study resources showed their own educational methods for a broader audience. Moreover, infrastructure is intended to attract large communities within educational systems. At this point, State of the Art review was conducted to explore ways to

improve the efficiency and functionality of an educational platform. Both advantages and disadvantages are identified by comparing systems. Therefore, State of Art facilitates the development of a practice-oriented interface that avoids the same mistakes after successful analysis. The following Chapter will provide the overview of the web system, which includes bridges with the client and the system, frontend programming languages, and result of desired UI of EasyAI.

1. Chapter 1 - Overview of Web Application

The present chapter are aware of the web application is discussed, including:

- Fundamentals of web applications,
- Client-server architecture,
- HTTP communication,
- Front-end technologies.

1.1 Web Applications

For 30 years, frontend coding has been a necessary role for humans. Like drawing something new which attacks people attention, there were variety versions of UI and UX made by humans. Moreover, each website or even UI and UX has a specific design, such as its own fonts, background colors, main design colors, etc. I believe that not only the design but also the functionality of the UI is paramount. The design alone helps the user to establish a connection with programs, but it is the functionality that makes the system truly usable. For example, in human psychology, each color represents different emotions and feelings, so designers choose color palettes accordingly. Red is rarely used in UI because it can strain the eyes and give an aggressive impression. Therefore, I think that one of the main aspects of UI design is choosing the right colors. Since user interface design is an important component of modern web systems, it is also necessary to understand the overall structure and operation of web applications.

The popularity of educational systems in web applications has increased significantly due to their accessibility and flexibility. End users prefer applications that support practicality on electronic devices. Web applications are good examples of it. Unlike desktop software applications, web applications do not require installation on individual devices and can be run through a web browser. This also enhances the overall user experience; learners can continue to study across multiple devices.

1.2 Frontend Technologies

The main goal of this stage is to select the needed programming languages for building the user interface. There are several front-end technologies commonly used for this purpose, including HTML, CSS, and JavaScript. They can be considered a powerful combination that allows developers to design a wide range of user interfaces and user experiences. HTML supplies the basic

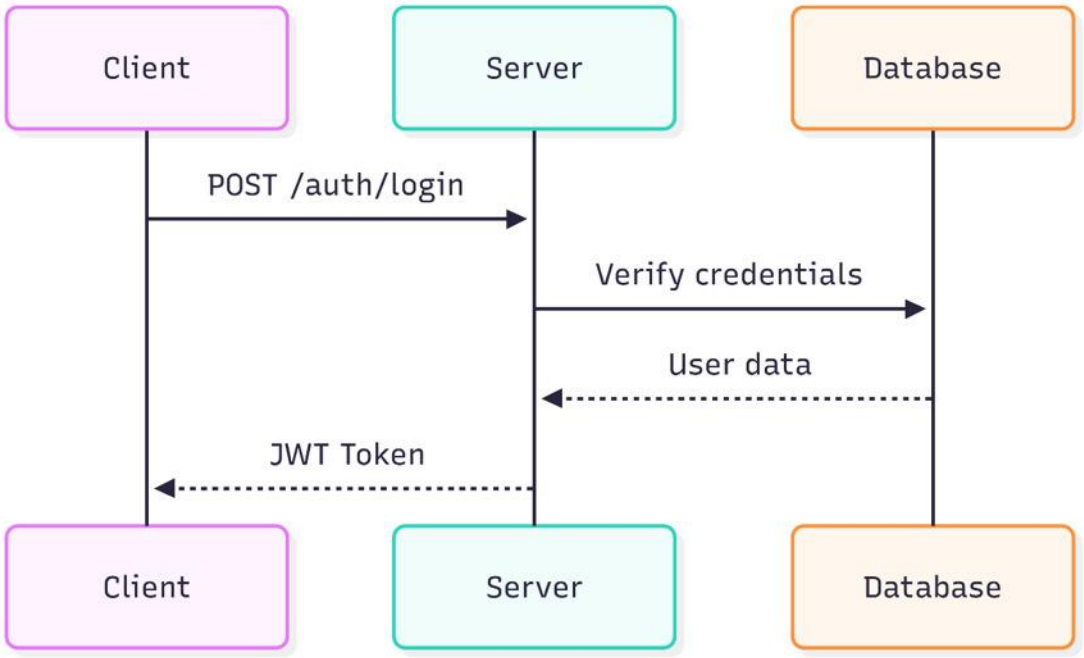
structure of content. CSS is widely used for essential visualization, such as various color palettes. Lastly, vanilla JavaScript provides various features like user input, animation, and data synchronization.

On the other hand, the React library is another popular option for building user interfaces. React suggests a different architecture and configuration. For example, it automatically applies some DOM changes using the virtual DOM mechanism. In addition, React relies on an ecosystem of build tools. These tools automate optimization and core configurations. Vite is the first one that comes to mind; it refreshes the browser instantly whenever the code is modified. However, plenty of disadvantages occur when not developing the planned platform. These can be called comparisons with current vanilla JavaScript. React requires extra installations before writing code. Developers must install Node.js, npm, Node.js modules, and build tools like Vite and more to enable its library compatibility. This causes the growth of code size, which can reach hundreds of megabytes. If the application is planned to deploy later, the vanilla JavaScript can be served as a single static file, while React requires an additional setup. In conclusion, both approaches offer their advantages. However, for the current scale of the EasyAI platform, the vanilla JavaScript approach is chosen due to its simplicity and lower setup cost.

1.3 Client-Server Architecture

This web application covers client-server architecture. Generally, clients stand for users interacting with the system, and the server stands for systems that work in the background. Client sends requests, server stores data, processes requests, and returns a response as soon as possible. The EasyAI platform also uses this architecture. The user interacts with the design and functionality of the system, which is built with HTML, CSS, and vanilla JavaScript, while the server uses the FastAPI framework for the backend. The client can send requests such as login, lesson generation, or chat messages. The server processes these requests, communicates with the SQLite database to store individual data and with the Ollama service to generate AI responses, and returns the result to the client in JSON format. Obviously, architecture is simple and easy to update separately if any errors occur. Even though the application runs locally, it supports multiple clients at the same time. Modern web applications usually use stateless architecture. However, the server remembers the client's previous request, so each request should be informative to be processed. EasyAI uses JWT tokens to solve this problem, which are sent with each request to identify the user. Below, it is described how client-server architecture works (see on Fig. 3.).

Fig.3. Diagram of Client-server architecture



Source: own study

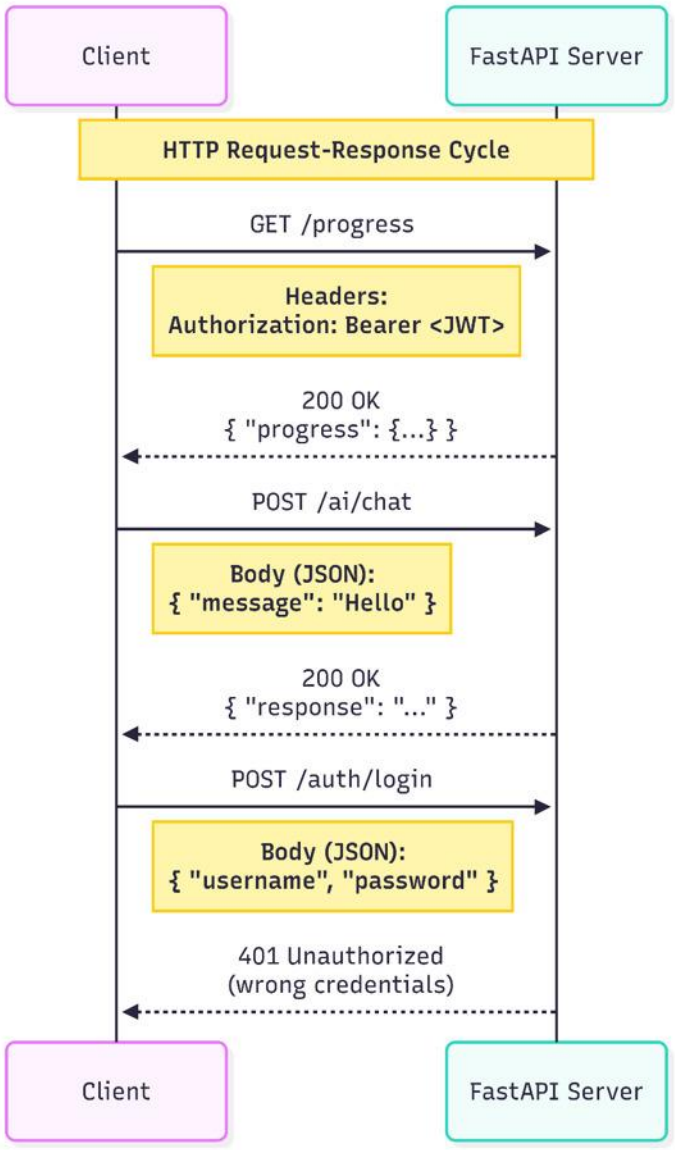
1.4 HTTP Communication

HTTP (Hypertext Transfer Protocol) is the standard protocol used for communication between web clients and servers. This Protocol is a bridge between the application and the internet. Also, most web applications use HTTP, including EasyAI. Obviously, HTTP works like that: clients send their requests, the server returns their responses. Each request contains a method, a URL, or headers. The response includes a status code that defines the situation of the running application and the requested data. The arrows between the web client and the router layer use HTTP or HTTPS to transport mostly JSON format. The arrows between the data layer and the database use a database-specific protocol and carry SQL (or other) text. The arrow between the layers themselves is a function call carrying data models.

The EasyAI system uses several HHTP methods. Mainly, the GET method is used to access data, such as loading the user's progress from the server. The POST operation is implemented to send data, for example, login information or the user's behavior in a lesson or a quiz. The rest of the techniques, like PUT and DELETE, are less common for end-users. Because the system architecture is offering to update data by these methods for the admin role. The status code is known

to define the result of the request. Code 200 means a successful answer, or 201 means a new resource is created, 401 code is used to inform that authorization is needed, and 500 code means a server error happened. EasyAI is planning to return these codes while developing the application. All data between the client and the server is exchanged in JSON format. And it raises the effectiveness of the application while sending data between the client and the server (see on Fig. 4.):

Fig. 4. Diagram describes the current HTTP Request-Response Cycle

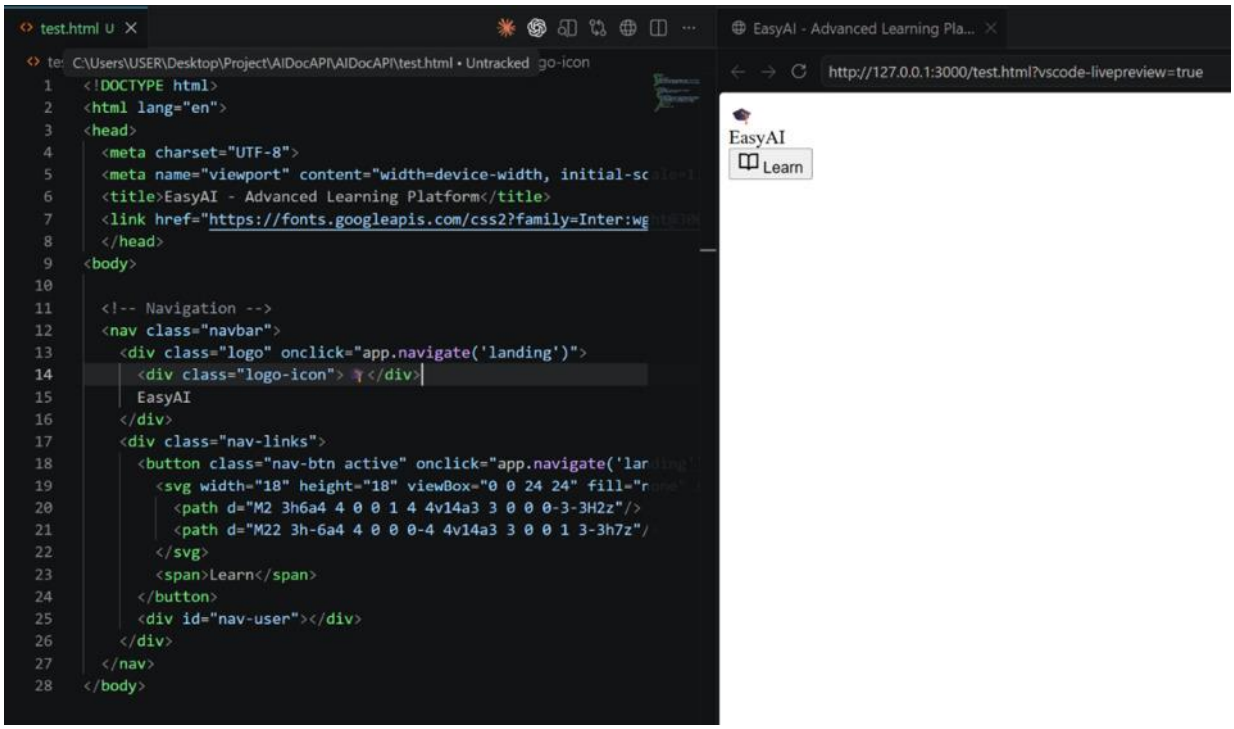


Source: own study

1.5 HTML

HTML (i.e., ***HyperText Markup Language***) is an essential scripting language for describing text formatting in a web browser. There is no alternative web language to HTML. Most web browsers must contain HTML tags to run properly. As Philip Ackermann describes “HTML: The first version (without a version number) was published in 1992 and was based on proposals that Tim Berners-Lee (the inventor of HTML) had already worked out in 1989 at the European Organization for Nuclear Research (CERN).” [1] The combination of HTML5, CSS3, and modern JavaScript (ES6+) is the standard approach for building responsive and accessible user interfaces today [1]. Namely, this language uses tags such as to assign elements, with closing slash symbols to close those tags. “The opening tag introduces an element, and the closing tag ends the element. For example, the opening tag <h1> introduces a first-level heading, and the (closing) tag </h1> ends the heading. As a result, the text between the two tags is the heading text.” [1] Each tag is defined to adjust a variety of visualizations. The main headings, paragraphs, containers, tables, and more are defined by these tags. Additionally, HTML allows developers to write CSS and JavaScript code in the same file. To achieve this, developers should add <style> and <script> tags. The slash symbol closes them too. Selecting proper tags will change the structure of the desired application (see on Fig. 5.):

Fig. 5. Developing core visualization of EasyAI

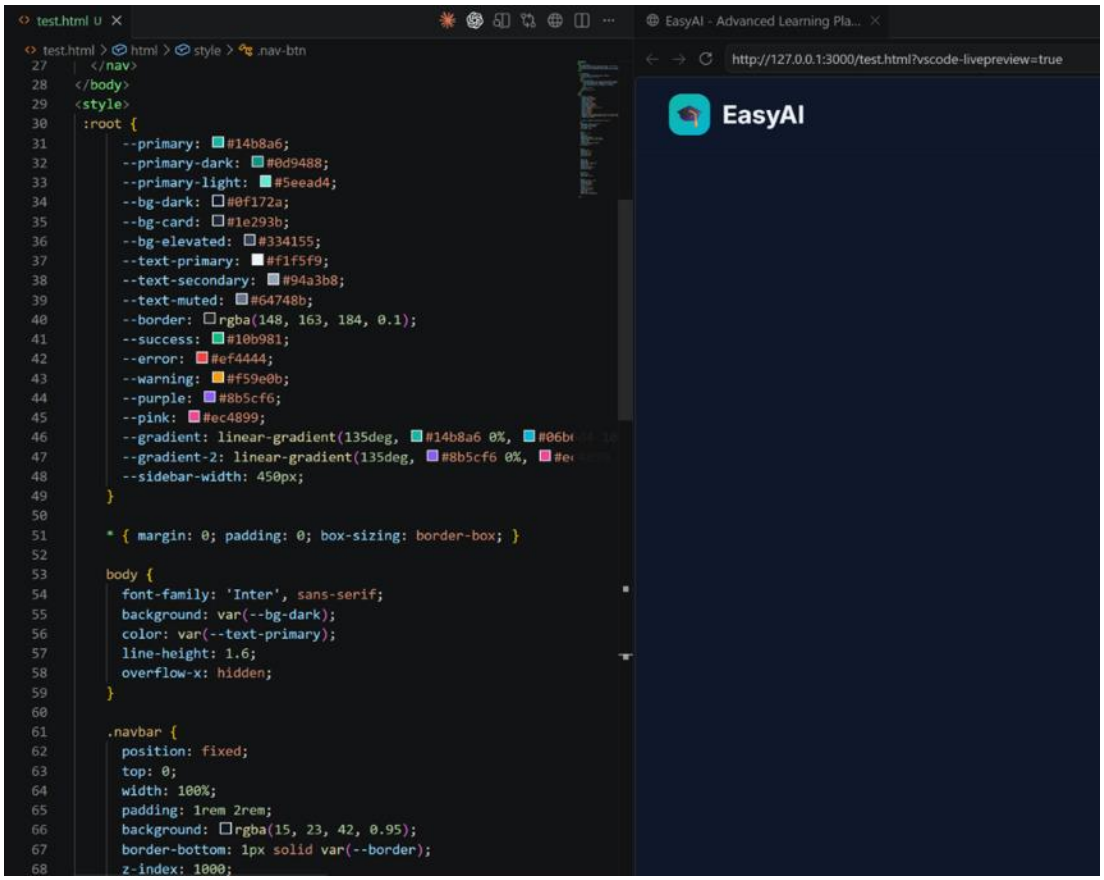


Source: own study

1.6 CSS

CSS (Cascading Style Sheets) is a style sheet language for text formatting, such as adjusting layout, colors, and fonts. “Cascading Style Sheets, usually abbreviated to CSS, is a style sheet language used to describe how to display documents written in HTML, such as web pages, and other markup languages. CSS works by implementing a series of rules that control the display of the elements that make up the web page.” [2] “When you need to apply specific formatting to individual elements in your documents, you can use inline CSS. Inline CSS essentially means creating a mini style sheet within a particular element, such as a heading or a list item. To implement inline CSS, you add the style attribute to the opening tag for the element and then specify the style formatting.” [2] Developers prefer to design each element to create their own interface. While developing applications, plenty of CSS versions are available as well. Such as Tailwind CSS, that enables developers to write all styling through classes. Developers use curly braces to design each element that is assigned in HTML. The design should be attractive to users. After adding some designs to the previous HTML code, the results will look like this (see on Fig. 6.):

Fig. 6. Result of CSS compared to the previous visualization

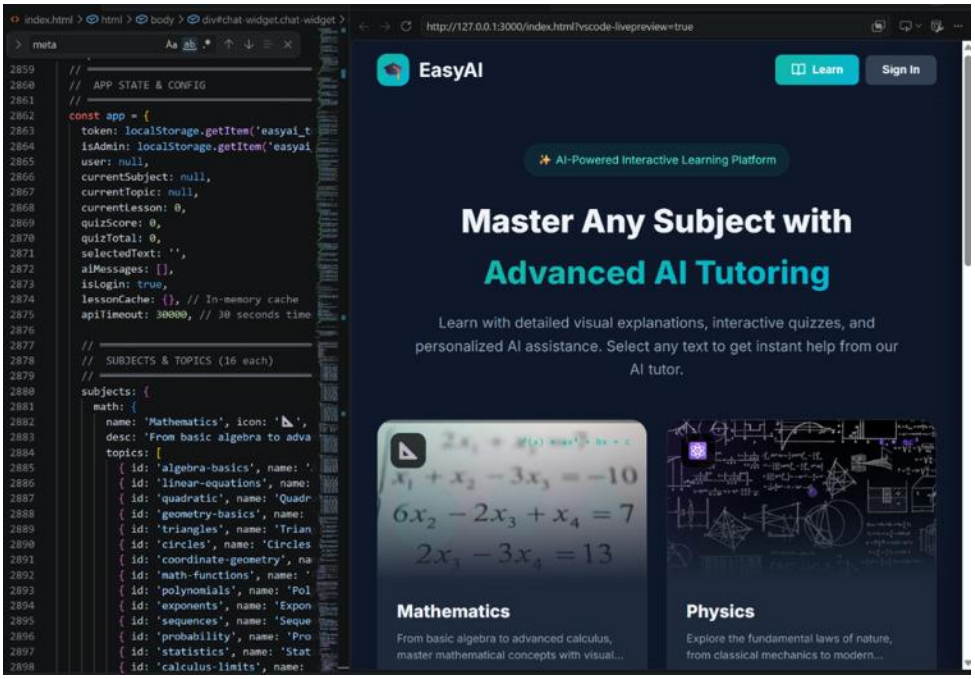


Source: own study

1.7 JavaScript

JavaScript is a programming language that is the core of web technologies besides HTML and CSS. While HTML defines the structure of a web page and CSS provides its visual appearance, JavaScript adds interactive and dynamic behaviors. “JavaScript is an interpreted programming language where the source code is executed by an interpreter on the computer. The interpreter, in turn, converts the source code into machine code, statement by statement. Usually, you don’t have to worry about anything. You can write the source code with any text editor, and the interpreter is provided and executed in the web browser.” [3] Without JavaScript, web pages will not respond to user actions. According to Wolf, “Client-side scripting languages are referred to as web-based scripts that are executed on the local computer, usually by the web browser. In common practice, JavaScript is the most significant client-side scripting language. JavaScript allows you to extend the limited capabilities of HTML with user interactions, for example, to evaluate, dynamically modify, and create content. Nowadays, no modern website can do without JavaScript.” [3] JavaScript enables input forms, animates elements, and handles button clicks. It creates a perfect relationship to communicate with the backend through an HTTP request, as mentioned in the previous subchapter. In EasyAI, JavaScript is at the central point of code to execute user actions such as selecting and opening a lesson, sending messages to the chatbot. Moreover, it can also send HTTP requests and receive JSON responses. Vanilla JavaScript enables all these functionalities without requiring extra frameworks or libraries. After writing JavaScript code, visual material will be changed like this (see on Fig. 7.):

Fig. 7. JavaScript change on the EasyAI platform



Source: own study

2. Chapter 2 - Selected Tech Stack(s)

This chapter targets to describe selected tech stacks which will play necessary role while building a responsive backend. Like Chapter 1, the focus of the subchapters is on presenting the core differences between two popular backend languages. Chapter 2 will include:

- Python vs JavaScript,
- FastAPI Framework,
- Open-access LLM Model – Ollama.

2.1 Python vs JavaScript

The first option to consider is Python. According to Matthes, “People use Python for many purposes: to make games, build web applications, solve business problems, and develop internal tools at all kinds of interesting companies. Python is also used heavily in scientific fields, for academic research and applied work.” [4] Also, Matthes emphasizes that “One of the most important reasons I continue to use Python is because of the Python community, which includes an incredibly diverse and welcoming group of people. Community is essential to programmers because programming isn’t a solitary pursuit. Most of us, even the most experienced programmers, need to ask advice from others who have already solved similar problems. Having a well-connected and supportive community is critical to helping you solve problems, and the Python community is fully supportive of people who are learning Python as their first programming language or coming to Python with a background in other languages.” [4] Lubanovic adds that “More recently, it’s become extremely popular in the data science and machine learning worlds. If you want to land a well-paying programming job in an interesting area, Python is a good choice now. And if you’re hiring, there’s a growing pool of experienced Python developers.” [5] Selecting the key programming language depends on the capabilities of that language. Python is one of the most widely used programming languages today. Developers benefit from its simple syntax and strong support for artificial intelligence. For instance, PyTorch and Ollama’s official client are written in Python. Python has been leading the field of artificial intelligence for 10 years. It is modern AI frameworks such as TensorFlow, PyTorch, and Hugging Face Transformers, provide core support within Python libraries. This rich ecosystem delivers accessible tools and models for developers. The EasyAI platform relies on local AI models to generate lesson content and quiz questions. Without Python’s ecosystem, the Ollama integration would demand significant effort to bring additional tools. Namely, Python can be considered a suitable programming language to provide lots of frameworks and libraries in the machine learning world.

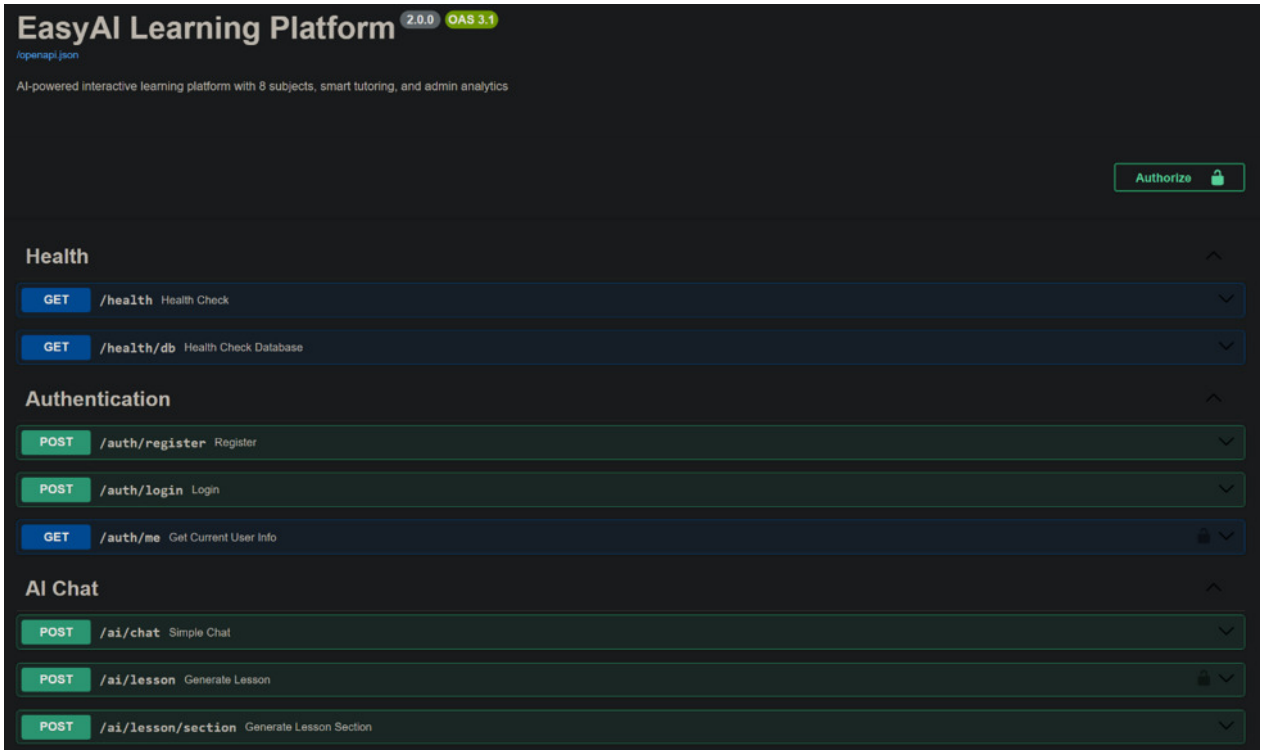
“The age of JavaScript is truly upon us. From its humble beginnings as a client-side scripting language, not only has it become completely ubiquitous on the client side, but its use as a server-side language has finally taken off too, thanks to Node.” [6] “Another major difference between Node and more traditional web servers is that Node is single-threaded. At first blush, this may seem like a step backward. As it turns out, it is a stroke of genius. Single threading vastly simplifies the business of writing web apps, and if you need the performance of a multithreaded app, you can simply spin up more instances of Node, and you will effectively have the performance benefits of multithreading.” [6] According to Casciaro and Mammino, “Some principles and design patterns literally define the developer experience with the Node.js platform and its ecosystem. The most peculiar one is probably its asynchronous nature, which makes heavy use of asynchronous constructs such as callbacks and promises.” [7] At first glance, it may seem that selecting either of these languages would not affect development. However, Python is better suited for this project compared to JavaScript. Python delivers a strong AI ecosystem which enables compatibility with Ollama. Since Ollama can generate AI responses locally, using Python and its libraries significantly facilitates the generation of lesson content. Obviously, JavaScript has enough potential to construct similar options. Yet Python remains the more practical choice because of its direct integration with the Ollama framework. As mentioned in Chapter 1, the Node.js environment requires a variety of configurations to set up. To sum up, choosing Python allows a suitable environment on the platform.

2.2 FastAPI Framework

A web framework is needed to handle HTTP requests and organize routes on the backend. In the Python ecosystem, several frameworks are commonly used, including Flask, Django, and FastAPI. On the web page, it is described that the framework supports asynchronous request handling and automatic data validation. Flash and Django are also popular Python frameworks. Flask is easy to learn, but it does not fully support asynchronous requests. Django provides complete features like ORM (Object-Relational Mapper), which creates its own database instead of using SQL, but it is not as simple as FastAPI. As Lubanovic explains, “Like any web framework, FastAPI helps you to build web applications. Every framework is designed to make some operations easier — by features, omissions, and defaults. As the name implies, FastAPI targets development of web APIs, although you can use it for traditional web content applications as well.” [8] Tragura also notes that “Frameworks that are easy to understand and use, seamless during coding, and within standards are always picked because of the integrity they provide to solve problems without risking too much development time. And a promising Python framework called FastAPI, created by Sebastian Ramirez, provides experienced developers, experts, and enthusiasts the best option for building REST APIs and microservices.” [9] Nevertheless, FastAPI balances simplicity and performance.

FastAPI is the best option for the EasyAI platform for several reasons. Firstly, it delivers responsive API call processing that facilitates generating AI responses quickly. Secondly, it allows the use of the Pydantic library to validate incoming data automatically. Lastly, FastAPI generates the needed API documentation through the Swagger UI endpoint, which makes testing and debugging easier. Swagger UI provides a clear overview of all those endpoints like authentication, lesson generation, and even quiz generation (see on Fig. 8.):

Fig. 8. Endpoints on Swagger UI



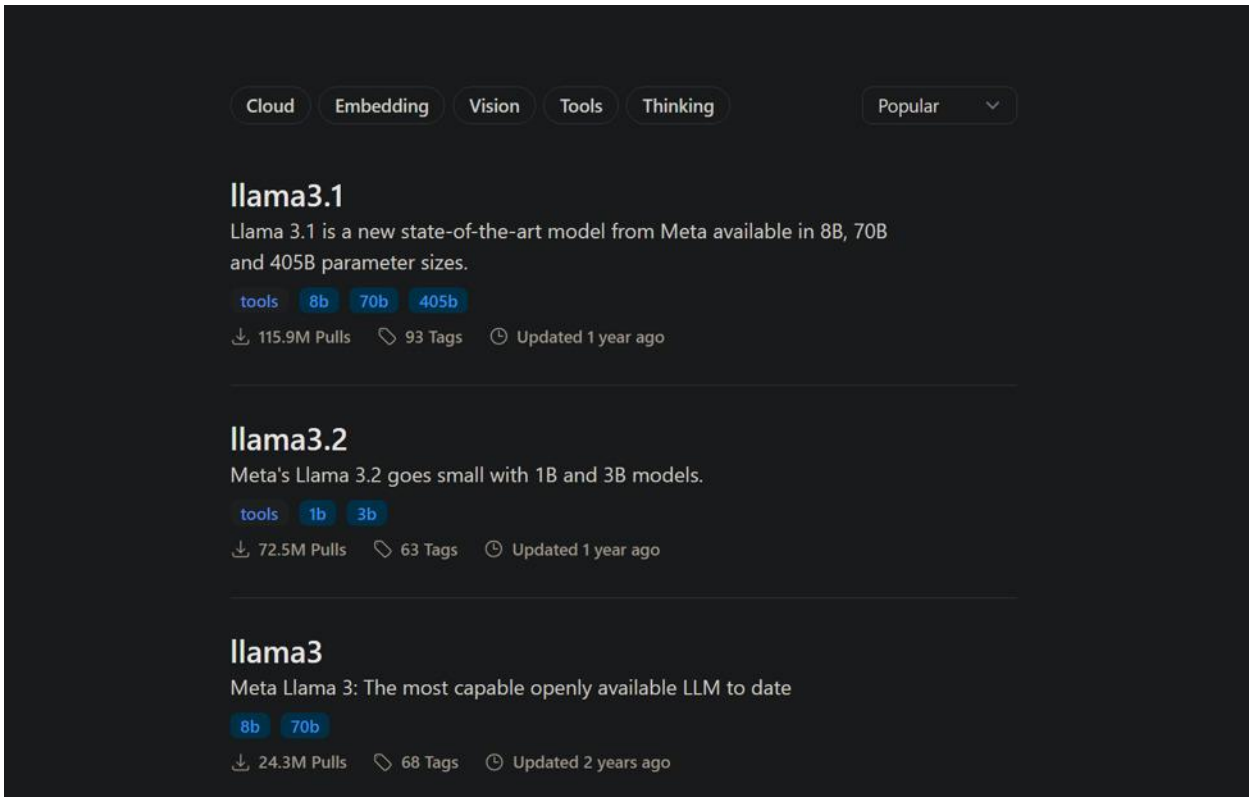
Source: own study

2.3 Open-access LLM - Ollama

Large Language Models are frequently used in AI systems to enhance the functionalities of these systems. Even users prefer to ask some essential questions on small chatbots that are located in the corner of the screen. Each developer uses Large Language Models for different purposes. Most LLMs are accessed through cloud services such as OpenAI GPT, Anthropic Claude, or Google Gemini. However, developers should allocate financial resources to use existing models. These services require paid API keys, which can be expensive for educational projects. In contrast, EasyAI uses Ollama models that deliver a variety of models at no cost, unlike existing models. Ollama is an open-source framework that runs LLMs directly on the user's computer. "Generative AI for text is now almost entirely focused on building large language models (LLMs), whose sole

purpose is to directly model language from a huge corpus of text—that is, they are trained to predict the next word, in the style of a decoder Transformer." [10] "The large language model approach has been adopted so widely because of its flexibility and ability to excel at a wide range of tasks. The same model can be used for question answering, text summarization, content creation, and many other examples because ultimately each use case can be framed as a text-to-text problem, where the specific task instructions (the prompt) are given as part of the input to the model." [10] The whole process occurs locally and offline, without paying for AI access. Ollama presents llama3.2:3b, qwen2.5-coder:7b, llama3.2:1b, and llama3.1:8b. Instead of using a single model for all tasks, EasyAI introduces a model selection mechanism. When a user sends a request, the system analyzes the content of the request, defines the complexity of the message, and then automatically chooses the most suitable model. For example, a user wants to ask a basic question to a chatbot. Then llama3.2:1b can reply in less than 5 seconds, while other models would take longer. The results below look like developer's terminal (see on Fig. 9.):

Fig. 9. Ollama models in a web browser

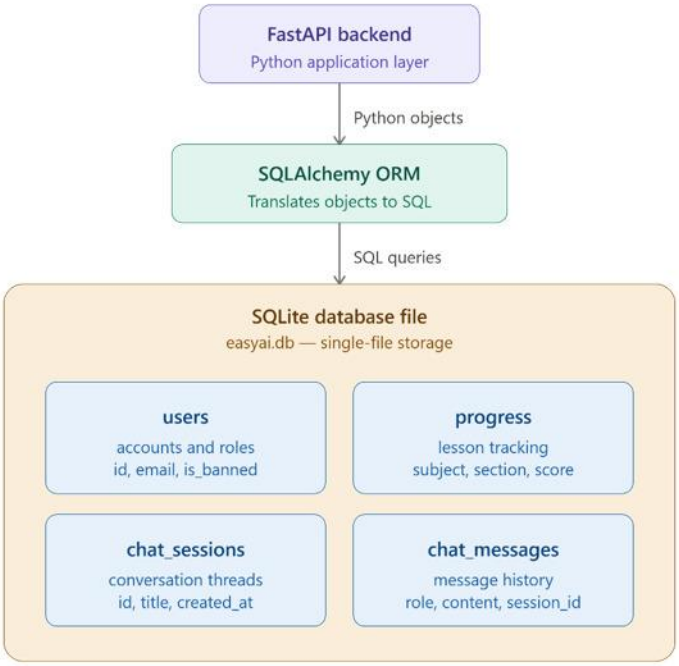


Source: https://ollama.com/search?q=llama, dated 20.05.2026

2.4 Database

A database is known as a collection of data for information management. Plenty of database engines are commonly used during the development of applications. For the EasyAI platform, the choice of the database engine plays an important role, since the application stores user records, progress records, and chat sessions. According to Coronel and Morris, “Databases evolved from the need to manage large amounts of data in an organized and efficient manner. In the early days, computer file systems were used to organize such data. Although file system data management is now largely outmoded, understanding the characteristics of file systems is important because file systems are the source of serious data management limitations.” [11] The most common ones include PostgreSQL, MySQL, MongoDB, and more. Most of them are used to manage data for large and complex applications. However, SQLite can be considered a more accessible option compared to popular database engines. The reason is that SQLite is a file-based engine that does not require a separate database server, which is suitable for a learning platform. Unlike other engines, SQLite stores the entire database in a single file, which makes development practical. The backend communicates with SQLite through SQLAlchemy ORM, which allows developers to interact with databases using Python objects instead of writing raw SQL queries. It is described as below (see on Fig. 10.):

Fig. 10. EasyAI database access flow



Source: own study

3. Chapter 3 – Preliminary specification – Functional and Non-Functional Requirements

This chapter will deliver the functional and non-functional requirements of EasyAI. Those requirements describe essential features that support the functionality of the system. Functional requirements cover specific behavior and function what the system does. EasyAI will describe features such as authentication, lesson and quiz generation, and AI components. Each requirement has a unique ID, like FR (functional requirements) or NFR (non-functional requirements). The Unified Modeling Language (UML) diagram provides a detailed overview to visualize the structure and behavior of an application. UML diagrams are divided into 2 parts: static diagrams that define components of the system, and behavior diagrams, which illustrate how those components interact with the system. For the EasyAI application, two types of UML diagrams will be presented in this chapter. The Use Case Diagram shows core interactions between users and the system, and the Class Diagram demonstrates database entities and the relationships between data entities. Everything will be divided into subchapters.

3.1 Functional Requirements

FR-01: **User Registration** – The system should allow new users to register by providing a username and password. When registration is submitted, the backend processes the input data, hashes the password using bcrypt, and stores the new user data in the SQLite database. After successful registration, a JWT token will be generated, and the system will confirm the user's session. Then the user can access the main page of the application.

FR-02: **Adjusting learning level** – Before generating lesson content, the system will provide learning levels such as Beginner, Intermediate, and Expert. Rather than targeting general audiences, the system delivers a different approach for users. Most educational platforms target beginner learners. However, offering diverse options would significantly improve users' experience in understanding lesson content easily. The difference between levels depends on the prompt for AI lesson generation by Ollama.

FR-03: **AI-Generated Quizzes** – Besides supplying the lesson level, the quiz component plays a necessary role in checking the client's knowledge when finishing the lesson. The user should answer all questions correctly. Then the application will mark that the topic is done. The number of questions depends on the content of the topics. Most topics contain 10 questions; some of them contain more. The most boring part is answering the same questions again. In a common case, the

user will fail and want to take the quiz again. The whole process starts to repeat, and the user may memorize questions if none of the content changes. To solve this, the system changes quiz questions after each attempt by the user. The user should consider that the quiz component is updating itself.

FR-04: **AI-Chatbot** – The platform will answer users' questions through a chatbot that is located in the corner of the screen. The user clicks the chatbot logo, and a small page appears that the user can interact with through text messages. The Chatbot provides a flexible interface that users can interact with anytime. If the user does not understand the explanation of topics, the system offers to users to copy any part of the explanation by highlighting and asking desired questions. The chatbot will automatically explain and focus on that problem.

3.2 Non-functional Requirements

NFR-01: **Response time** – AI response will not exceed 30 seconds for most interactions. The defined model selection boosts performance by relying on feedback that depends on the user's request. Performance is of great importance since it affects the quality of the user experience. EasyAI will provide a performance platform where users will not wait for minutes to access lessons or quizzes. Speedy request handling of FastAPI also supports the performance of the system.

NFR-02: **Local Processing** – The application keeps all data locally. Instead of covering the cost of a cloud-based API, Ollama responds to the entire AI process quickly. Then Ollama protects sensitive information, and there is no need for AI providers such as OpenAI and Claude.

NFR-03: **Reliability** – As an AI-powered educational platform, EasyAI tracks clients' data to improve user experience when generating new quiz questions. The platform will be enabled to receive wrong answers and change the same question without altering the content of the explanation. However, users can ask a specific quiz question if the question is not clear enough. The chatbot aims to resolve any unclear information in the application.

NFR-04: **Usability** – The interface follows familiar designs: a navigation bar at the top, informative subject cards, and an accessible chatbot. The back button will appear in lessons or quizzes to give

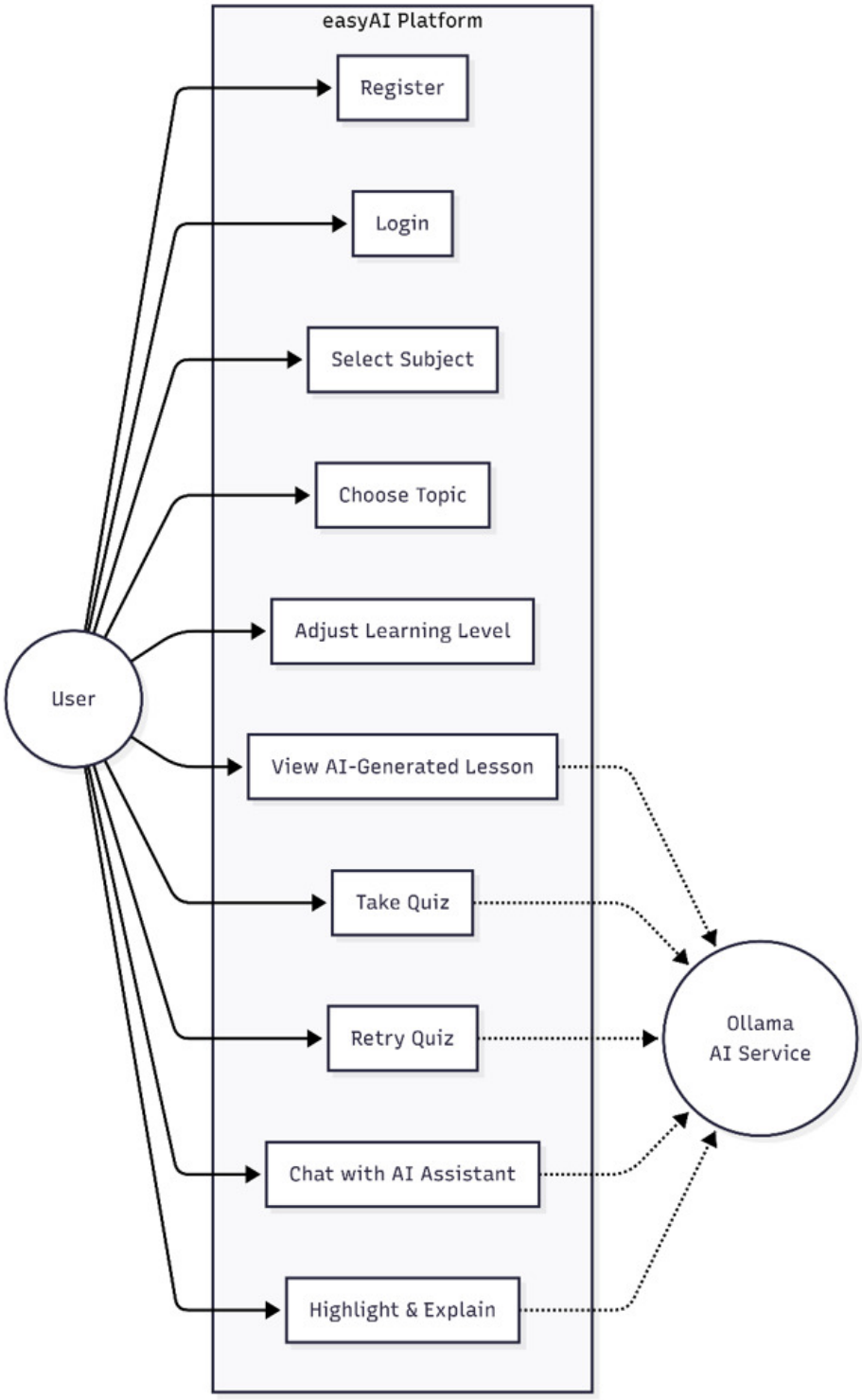
users the chance to save their progress before leaving the page. All saved progress will be stored in the backend. The system ensures that the focus of users is on the lesson content and its features.

3.3 UML

According to Washizaki, “The unified modeling language (UML) recognizes a rich collection of modeling diagrams. These diagrams, along with the modeling language constructs, are used in two common model types: structural models and behavioral models.” [11] “Behavioral diagrams provided by the UML for behavioral modeling include use case, activity, state machine, and interaction (sequence, communication, timing, and interaction overview) diagrams.” [11] “A use case is a list of actions or event steps typically defining the interactions between a role of an actor and a system to achieve a goal. A use case is a useful technique for identifying, clarifying, and organizing system requirements. A use case is made up of a set of possible sequences of interactions between systems and users that defines the features to be implemented and the resolution of any errors that may be encountered.” [12] “A Class is a blueprint for an object. Objects and classes go hand in hand. We can't talk about one without talking about the other. And the entire point of Object-Oriented Design is not about objects, it's about classes, because we use classes to create objects. So a class describes what an object will be, but it isn't the object itself.” [13] “In fact, classes describe the type of objects, while objects are usable instances of classes. Each Object was built from the same set of blueprints and therefore contains the same components (properties and methods). The standard meaning is that an object is an instance of a class and object - Objects have states and behaviors.” [13]

Use case diagram – The use case diagram of the EasyAI platform has two main actors: the User and the Ollama AI Service. The first actor, User, refers to any registered or unregistered person who interacts with the platform within the web interface. The second actor, Ollama AI Service, is a secondary system actor that assists in running the AI-powered backend. The user access actions include registering, selecting a topic, and adjusting the learning level. And the Ollama AI Service acts as an external actor that generates lesson content and quiz questions. This type of diagram is used to describe the perfect connection between the client and the server architecture as visual material. Although the user does not interact with Ollama directly, the user plays the key role in running an application. These are described below (see on Fig. 11.):

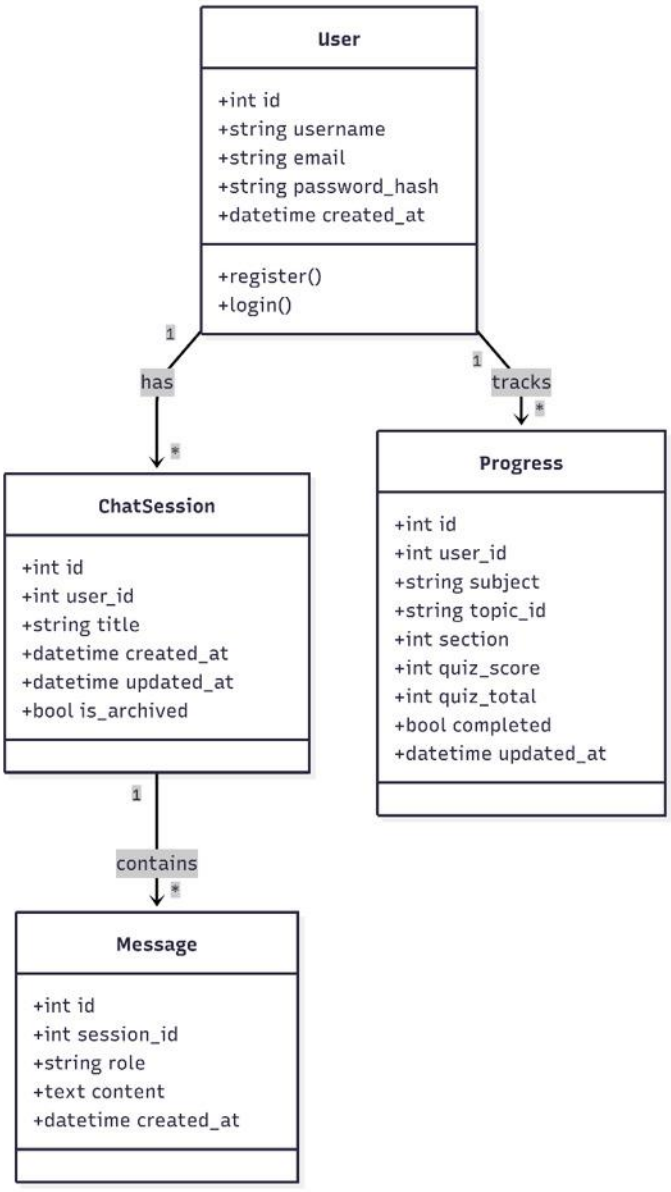
Fig. 11. Use case diagram of the EasyAI platform



Source: own study

Class Diagram – The class diagram represents four main entities that create the desired environment on the platform. Each entity has its own attributes and a unique ID. Besides attributes, there are relationships between entities. For example, each user tracks one progress, and a lot of progress can be tracked by one user. Some of the entities support their own data or functions, which are described in the diagram. The diagram presents an essential visualization of the internal data structure, with which they can interact with those functionalities. While the Use Case Diagram describes user-facing functionality, the Class diagram represents how information is transmitted inside the application (see on Fig. 12.):

Fig.12. Class diagram of the EasyAI platform



Source: own study

4. Chapter 4 – Backend Architecture and Implementation

From Chapter 1 to Chapter 4, an overview of the system was described, including selected frontend programming languages, backend frameworks, and main functionalities. However, this chapter focuses on the business logic of EasyAI, which describes the internal organization of the backend – including the role of controllers, routers, and service layers. The goal of this chapter is to present system integration with incoming requests, business operations, and connection with the database. This internal organization will demonstrate the effectiveness of the whole architecture while users sign up and start generating lessons. This chapter will state:

- Layered architecture overview,
- API layer,
- Data access layer.

4.1 Layered architecture overview

The backend of EasyAI follows a three-layered architecture, where each layer takes clear responsibility for running a proper system. This layered architecture allows developers to make any changes without extra modifications to the others. The first layer is the API, which provides controllers and routing. This layer accepts all incoming HTTP requests sent by the user. This layer is responsible for receiving requests, detecting the parameters of the URL, and sending data to the proper service. The layer implements the FastAPI router, which is related to endpoints. Each router has specific missions such as authentication, chatbot operations, and progress tracking.

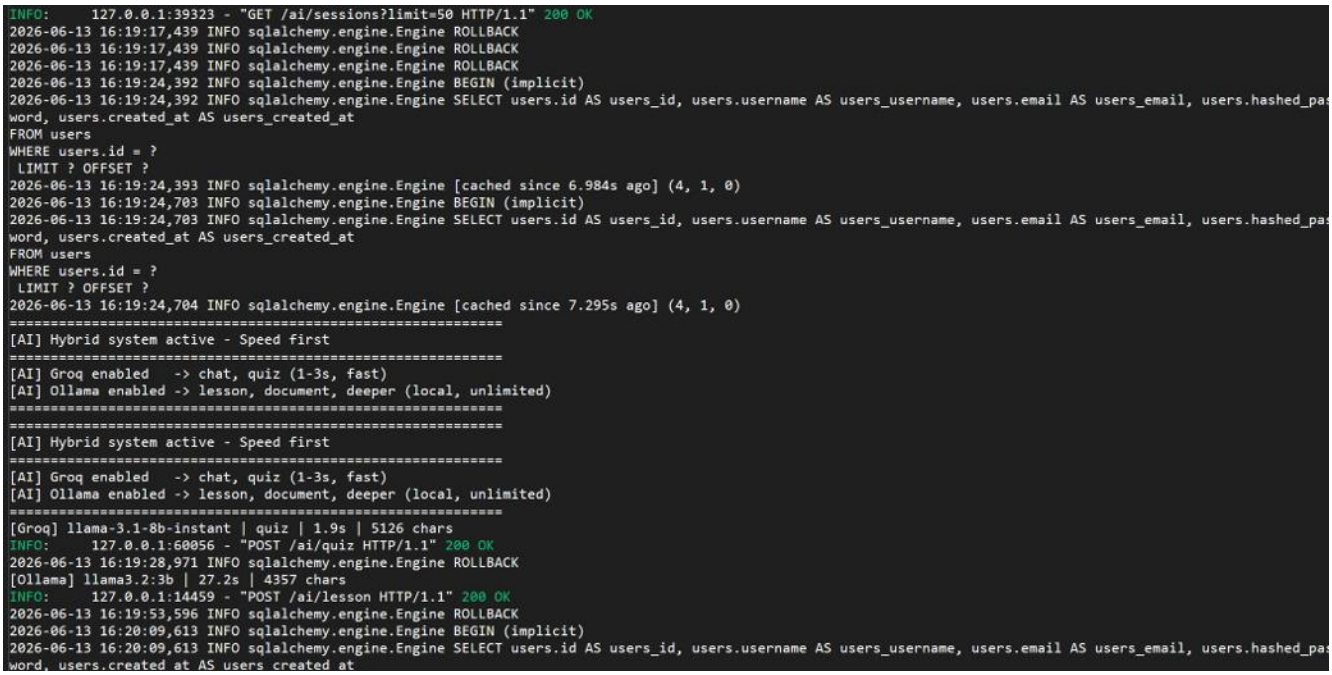
The second layer is the service layer, which defines the concept of the business logic of the system. This layer controls what and when the system does. When the API layer sends requests, the service layer processes them by building AI prompts and transforming data between components. For example, when a user clicks on any topic, the lesson service selects a suitable AI model, such llama 3.1 or llama 3.2, generates a result for the user, and delivers it to the canvas.

The third layer is the data access layer, which manages interactions with the SQLite database. This layer uses SQLAlchemy ORM (Object Relational Mapper) to place user requests in the database table. On the other hand, SQL is another option; however, SQLAlchemy is more compatible with SQLite. Developers can interact with SQLAlchemy through Python objects rather than writing raw SQL queries.

Overall, the layered approach enables each layer to depend on the others. The API layer depends on the service layer, and the service layer depends on the data access layer. This dependency delivers clean architecture, and it allows each layer to be adjustable. For example, the database engine can be changed from SQLite to PostgreSQL; it will not require any extra adjustments on the other layers.

The lesson generation flow is great proof to demonstrate the three-layered architecture of EasyAI. The frontend sends a POST request to the /ai/lesson endpoint with the topic and level information. Later, the API layer receives requests and validates the user's data. The service layer decides to select the most appropriate AI model based on the topic and complexity and sends it to the AI model. AI model generates the content of the lesson and sends it back to the service layer. The service layer analyzes the response and returns it to the frontend in JSON format. This process is described briefly like this (Fig. 13.):

Fig. 13. The whole mechanism of lesson generation on Developer PowerShell



Source: own study

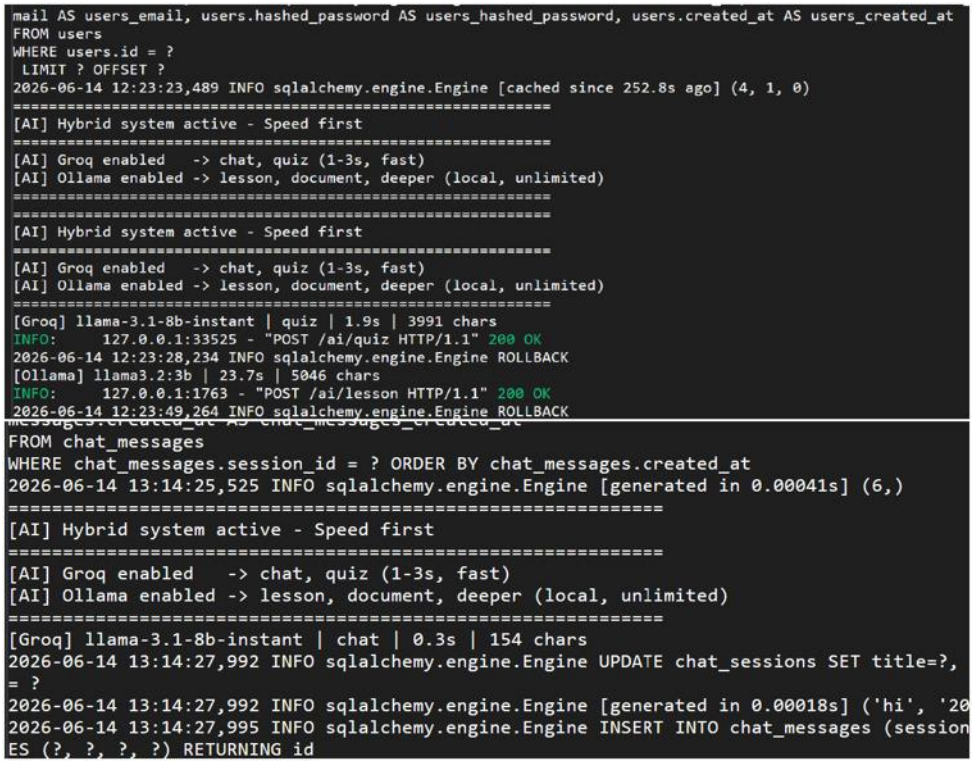
4.2 API layer

The API layer of the EasyAI platform is implemented through FastAPI routers, where router modules are placed independently to provide a specific feature area. This modular design simplifies debugging of the platform. There are six API modules: auth.py, admin.py, ai.py, health.py, programming.py, and progress.py. Among these, ai.py is a key module of the platform, which contains the main endpoints of the EasyAI application.

Ai.py – AI integration module

The ai.py module can be considered the main component that manages interactions with AI models. It contains endpoints for lesson generation (/ai/lesson), quiz generation (/ai/quiz), and the chatbot (/ai/chat). Firstly, this module chooses the most suitable AI model based on the content of the request. Since the user can start lesson generation for algebra fundamentals or ask a basic question to the chatbot, both require different AI models because getting a response from the chatbot with limited tokens should be faster rather than lesson generation. Namely, either the Groq cloud service or local Ollama responds to the user's request. Later, the layer forwards data to the frontend in JSON format. The picture explains this structure by providing both lesson generation and chatbot response (see on Fig. 14.):

Fig.14. Lesson generation and chatbot response with Groq and Ollama



Source: own study

4.3 Data access layer

The data access layer is the last layer of the backend architecture. This layer ensures data is stored in the database. There are many models to store data and define relationships between them. All models are placed in one file with different class names. For example, the User class stores the email, username, and hashed password of each registered user. One of the models differs from the other system. The Progress model tracks the user's learning state during each lesson. Its importance lies in the fact that this model enables the system to be responsive to the users. The user may leave lessons anytime, and the system should save the user's progress to improve the user experience. Some of the entities are stored as strings, such as subject, topic ID, and lesson level. While the others are stored as integers, only one attribute (is_completed) is stored as a Boolean value. The Progress entity is described in the codebase like this (see on Fig. 15.):

Fig. 15. "Progress" model in the codebase

```python
# ═══════════════════════════════════════════════════════
#  PROGRESS
# ═══════════════════════════════════════════════════════
class Progress(Base):
    """
    User progress tracker — one row per (user, subject, topic).
    """
    __tablename__ = "progress"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)

    subject = Column(String(50), nullable=False)        # e.g. "math", "biology"
    topic_id = Column(String(100), nullable=False)      # e.g. "algebra-basics"

    level = Column(String(20), default="intermediate")  # beginner | intermediate | expert
    current_section = Column(Integer, default=0)
    total_sections = Column(Integer, default=0)

    quiz_score = Column(Integer, default=0)
    quiz_total = Column(Integer, default=0)

    is_completed = Column(Boolean, default=False)

    created_at = Column(DateTime, default=datetime.utcnow)
    last_accessed = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
    completed_at = Column(DateTime, nullable=True)

    # Each user can only have ONE progress row per (subject, topic)
    __table_args__ = (
        UniqueConstraint('user_id', 'subject', 'topic_id', name='uq_user_subject_topic'),
    )
```

Source: own study

5. Chapter 5-Presentation for End User and Admin role.

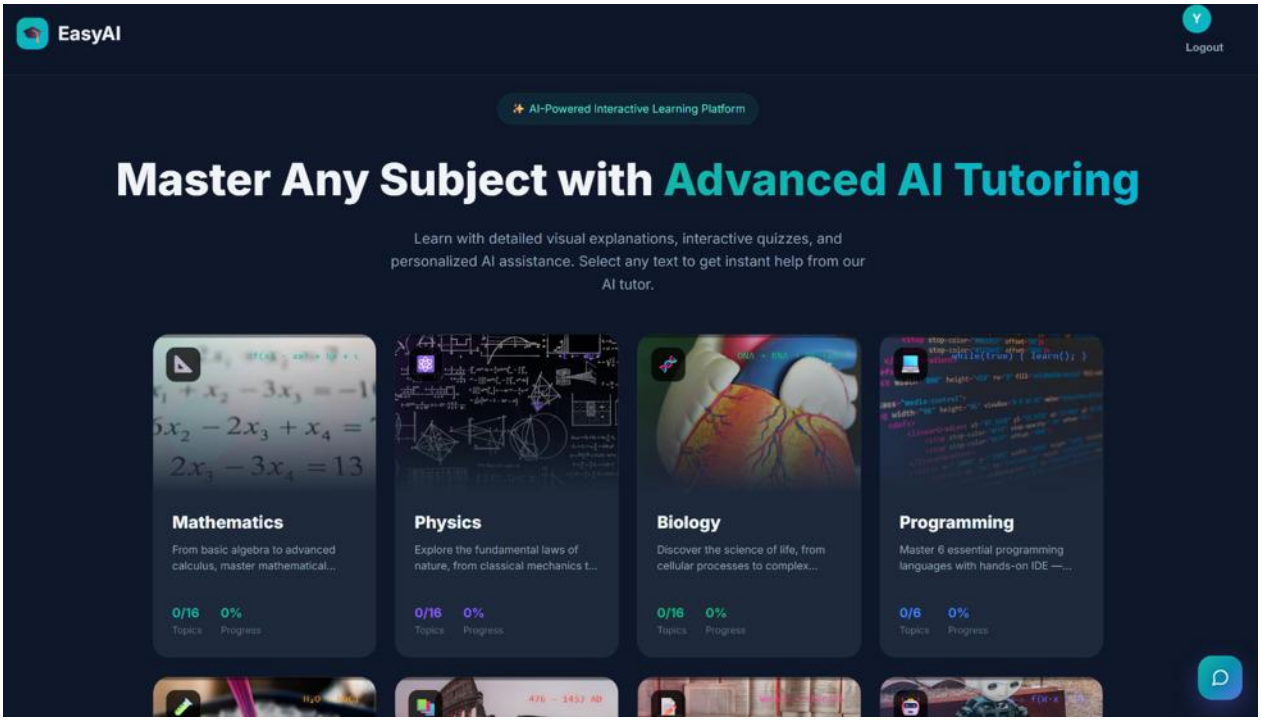
This chapter aims to present the user interface and overall visual materials of the AI-powered educational platform through screenshots. Unlike previous chapters, which focused on system architecture and backend implementation, this chapter will demonstrate how the system will be experienced from the user's perspective. Moreover, the system will provide additional features to the Admin, such as an admin panel to adjust the implementation.

5.1 Normal user

5.1.1 The landing page

The landing page delivers a responsive interface of the EasyAI platform. Firstly, it describes the overview of the platform. Later, this page offers eight subject cards such as mathematics, physics, biology, programming, chemistry, history, english, and AI & machine learning. In the bottom right of the screen, the chatbot logo appears after the user logs in (see on Fig. 16.):

Fig. 16. The landing page of the EasyAI platform



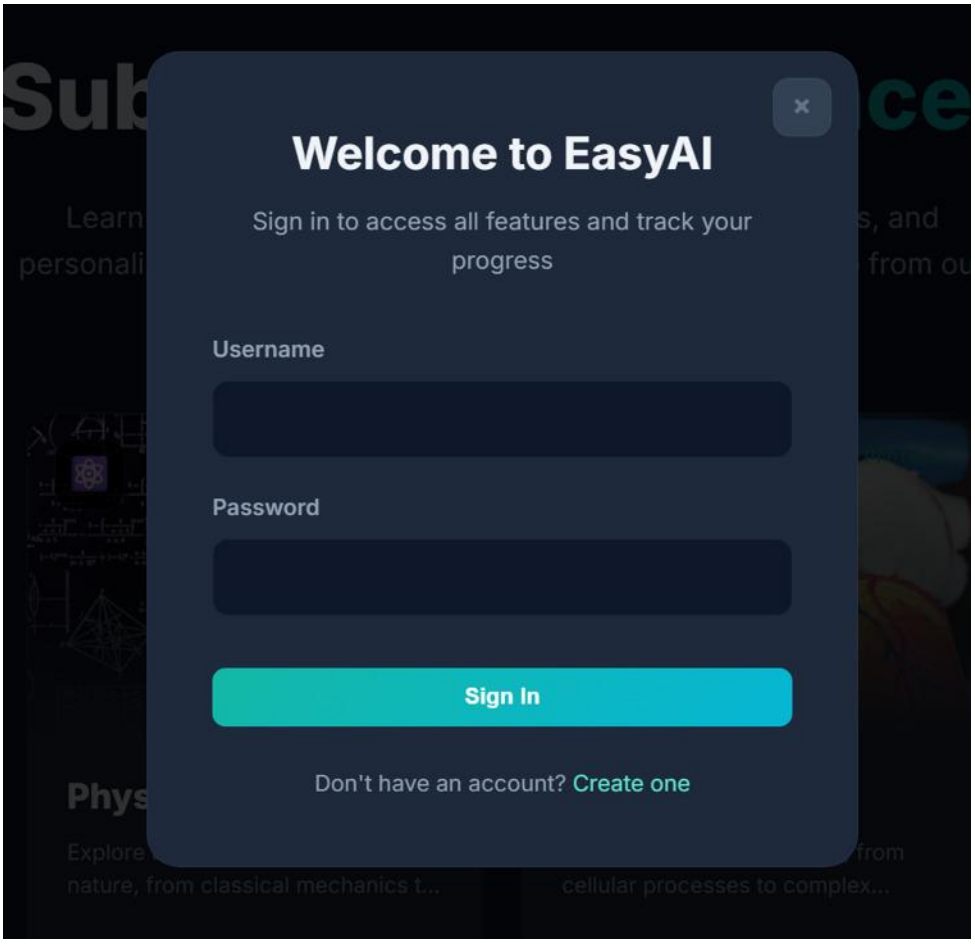
Source: own study

And the user can log out; however, the system will require authentication to track the user's data. The design describes an overview of the EasyAI platform with a clear and responsive frontend.

5.1.2 The Authentication page

Users should sign in or register by entering their credentials to access the educational platform. For registration, the user provides a username, an email, and a password, while for login, only the username and password are required. Then the backend validates the input, hashes the password, and stores data in the SQLite database. The email field is mandatory during registration, since it is later used by the administrator for account management. Authentication is obligatory to track users' data like progress of each topic, quiz attempts, and chatbot history (see on Fig. 17.):

Fig. 17. Authentication page of the EasyAI platform



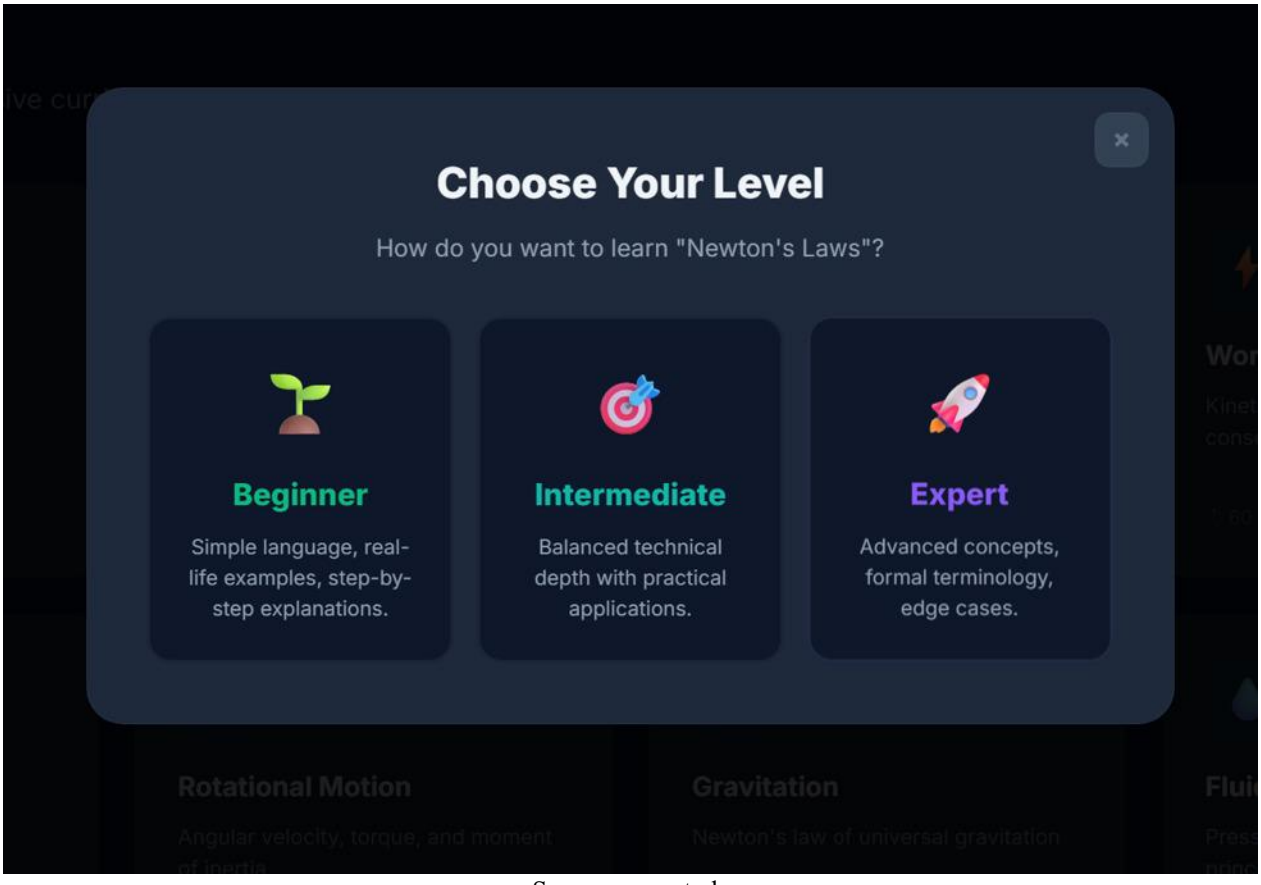
Source: own study

The same username is not acceptable in the database. Each username is unique to define the user's registration. After successful login or registration, users can text with chatbot or start any lesson of the subject. Besides the end user, the system delivers access to the admin mode. By logging in with credentials, admin access to the admin panel is provided to preview the user's data.

5.1.3 Learning Level Section Page

Before generating a lesson, the system presents a level section for the user. This section balances learning concepts by delivering a responsive approach. Most platforms target beginner learners and design their content for that level. However, the user would like to personalize the teaching approach according to their current knowledge level. Each level affects changes in the prompts of AI models. The system stores only the progress of the topic (see on Fig. 18.):

Fig. 18. Selecting the learning level on the EasyAI platform



Source: own study

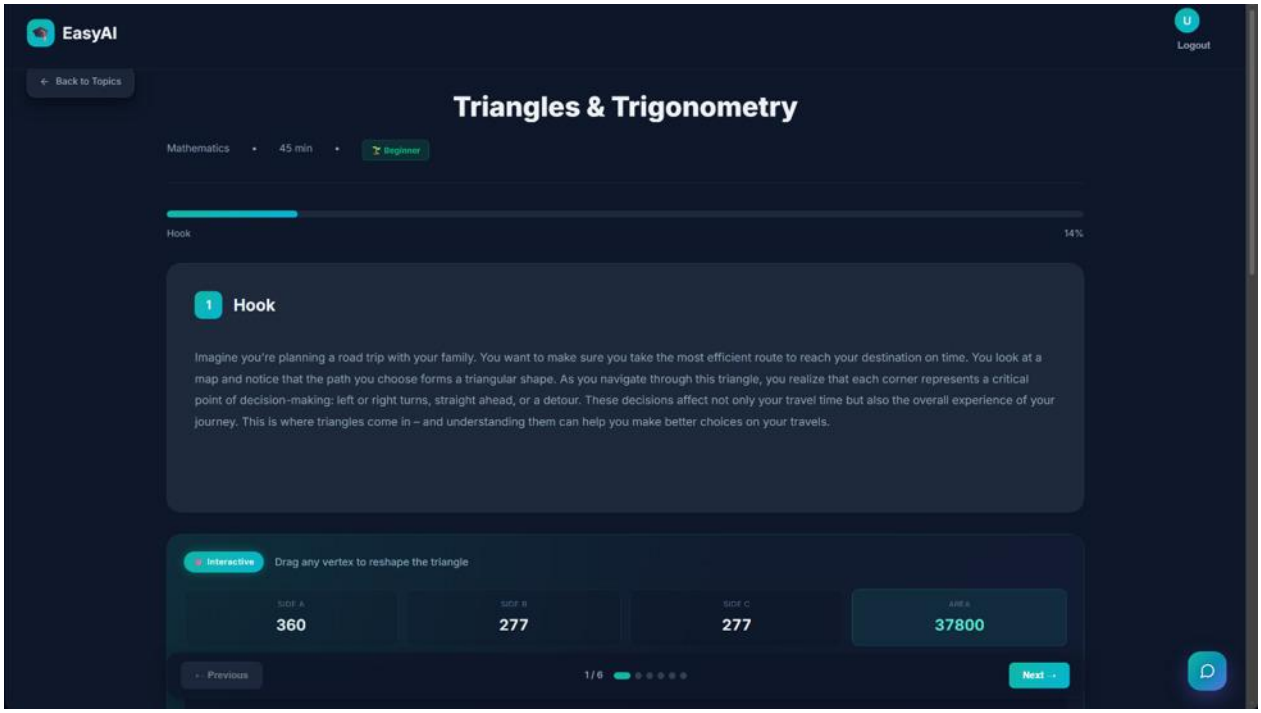
The reason is that the user should finish one of those levels to improve their knowledge. Since each level offers different approaches, selecting any level will not improve user experience or

knowledge efficiency. Moreover, the system marks it as complete when the user finishes one of the levels. Users can revisit the same topic to continue their progress.

5.1.4 Lesson Interface

Once the learning level is selected, lesson generation starts by using the backend architecture described in Chapter 4. At the top of the page, the lesson title, subject name, and selected level are clearly displayed. Below the header, a progress bar shows the user's current position within the lesson, divided into multiple sections. The main area presents lesson content, which is generated by an AI model, providing examples of lesson content. A sticky navigation bar at the bottom of the page allows the user to move between sections using "Previous" and "Next" buttons (see on Fig. 19.):

Fig. 19. Lesson interface on the EasyAI platform



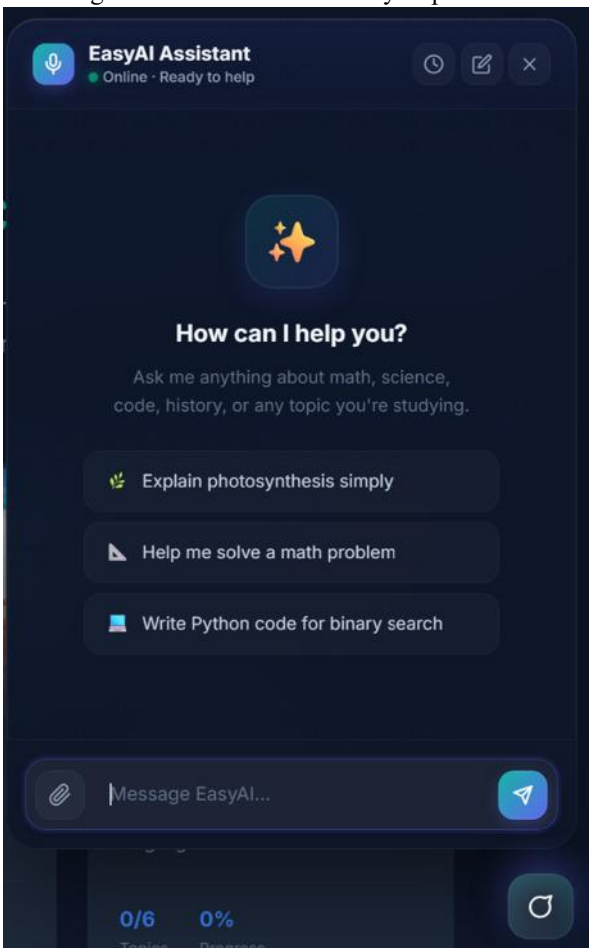
Source: own study

The system provides interactive components that allow the user to drag any vertex to reshape the triangle with the mouse. Each lesson includes an interactive component, which provides a variety of visual materials. At the end of the page, each lesson includes a YouTube video related to the topic. By that, the system provides plenty of video sources to improve knowledge efficiency.

5.1.5 AI Chatbot Assistant

The AI chatbot assistant is available on every page of the EasyAI platform. The logo consists of a flexible button that is in the bottom right corner of the screen. When the user clicks the button, a small window appears, providing an interface for interaction with the AI model. Its design allows users to ask questions on any page (see on Fig. 20.):

Fig. 20. AI chatbot of the EasyAI platform



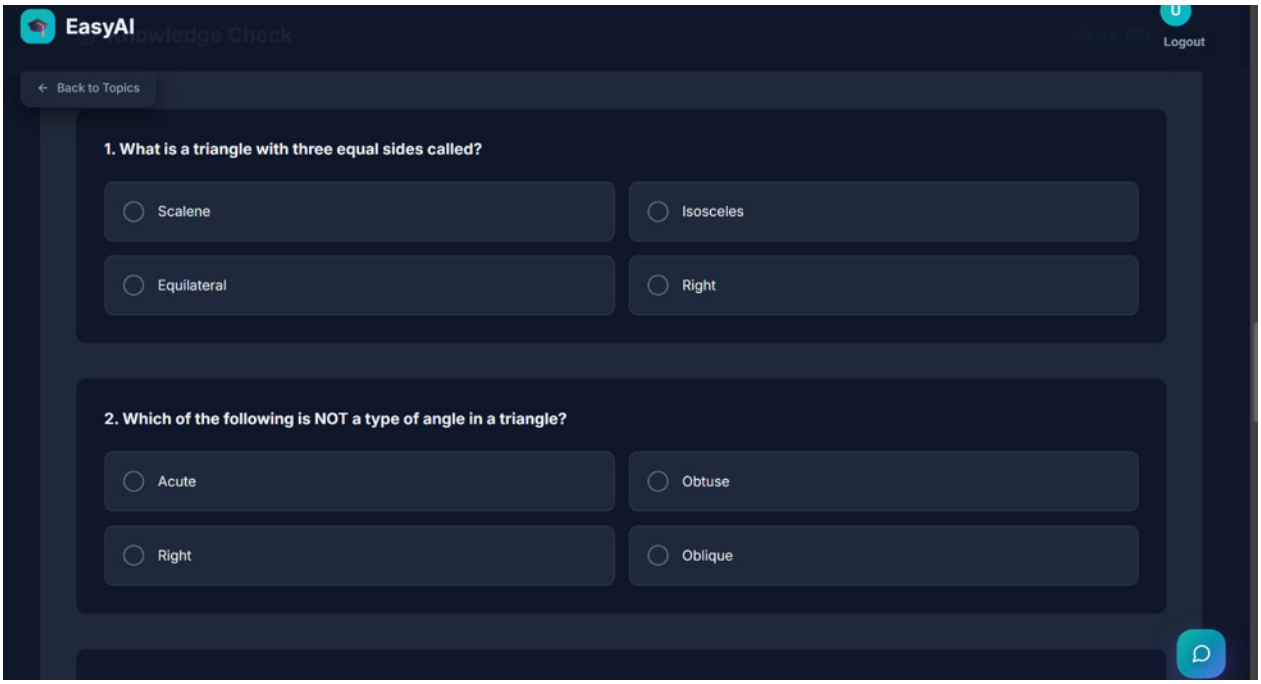
Source: own study

The user can see message history, upload documents, and open a new chat by clicking the logo on the interface. By uploading documents, users can get more accurate responses to problems. The system uses Groq models to get fast responses. Besides asking questions, the chatbot is helpful to explain your wrong answers on quiz questions.

5.1.6 Quiz Interface

After completing all lesson sections, the user proceeds to the quiz interface. The quiz is automatically generated by the selected AI model. The number of quiz questions depends on the content of the lessons. Some topics are more complex than others; the system ensures that the user understands the whole content. That is why the number of questions can be from ten to fifteen. Each question presents four answer options. When the user selects the answer, the system immediately gives feedback by highlighting the correct answer in green and wrong ones in red (see on Fig. 21.):

Fig. 21. Quiz Interface on the EasyAI platform



Source: own study

If the user selects the wrong answer, the chatbot opens itself and explains “why your answer is wrong”. With this functionality, users can ask questions about anything during the lesson. After answering questions correctly, the system marks the topic as done. The system requires the user to answer all questions correctly. Giving correct answers to all questions ensures that the user has learned the topic with the interactive component and video source.

5.2 Admin role

While the EasyAI platform focuses on delivering a functional learning application, the platform supports an administrative ecosystem to track users' data and monitor the system's status. This strategy requires a private username and password to authorize as the administrator. To keep the system secure, the admin role is added during the development of the application. Normal users cannot log in or access admin functionalities through the system. The authentication and security workflow operates by following these steps:

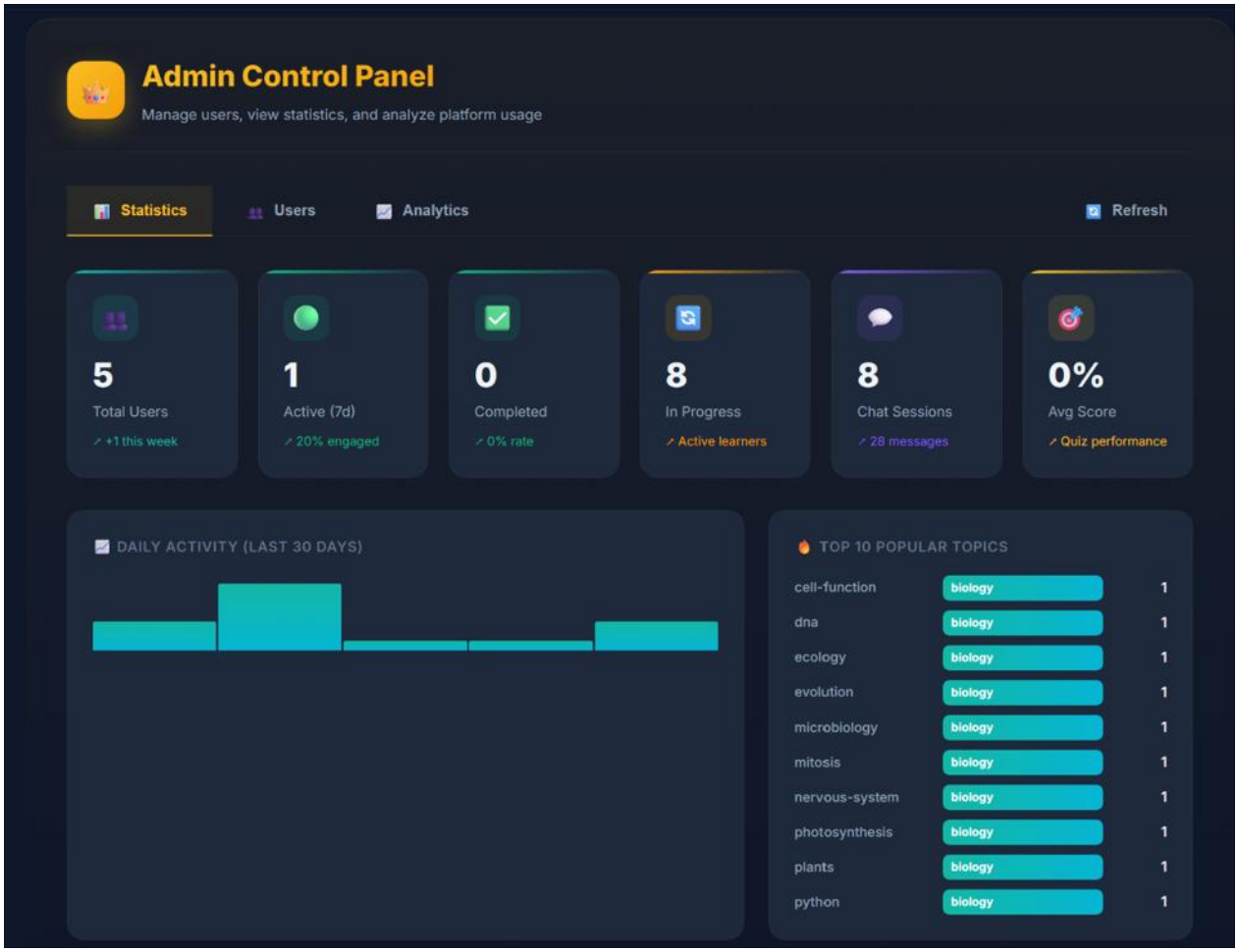
- Database verification: Since the system runs locally, the backend server receives the request and confirms its existence. If the needed data does not exist in the database, the error message appears immediately.
- Endpoint protection: The backend protects administrative data endpoints. Even if normal users try to access the admin page in the browser, the running server blocks the request and keeps the token private in JSON format.
- User management: The administrator can manage user accounts directly through the admin panel, including the ability to edit user details, reset passwords, and ban any user.

5.2.1 Admin panel

The admin panel is a special dashboard that provides several tools to manage the platform. It is called the "Admin Control Panel". These tools support managing users, viewing real-time statistics, and analyzing the performance of the system. The admin panel is organized into three main sections: Statistics, Users, and Analytics. Each separate tab helps to focus on specific areas of the application.

Statistics tab presents a real-time overview of the platform's activity. At the top of the section, six cards display key metrics: the total number of users registered on the platform, the number of Active Users during the seven days, the count of the Completed Topics, the number of topics currently in Progress, the total number of Chat Sessions, and the Average Quiz Score of all users on the platform. Below these cards, a Daily Activity chart is placed to show user activity within the last 30 days as a bar chart, while the Top 10 Popular Topics panel lists the most frequent topics across all subjects. Together, these sections provide a wide overview of how an administrator can track the system in real time (see on Fig. 22):

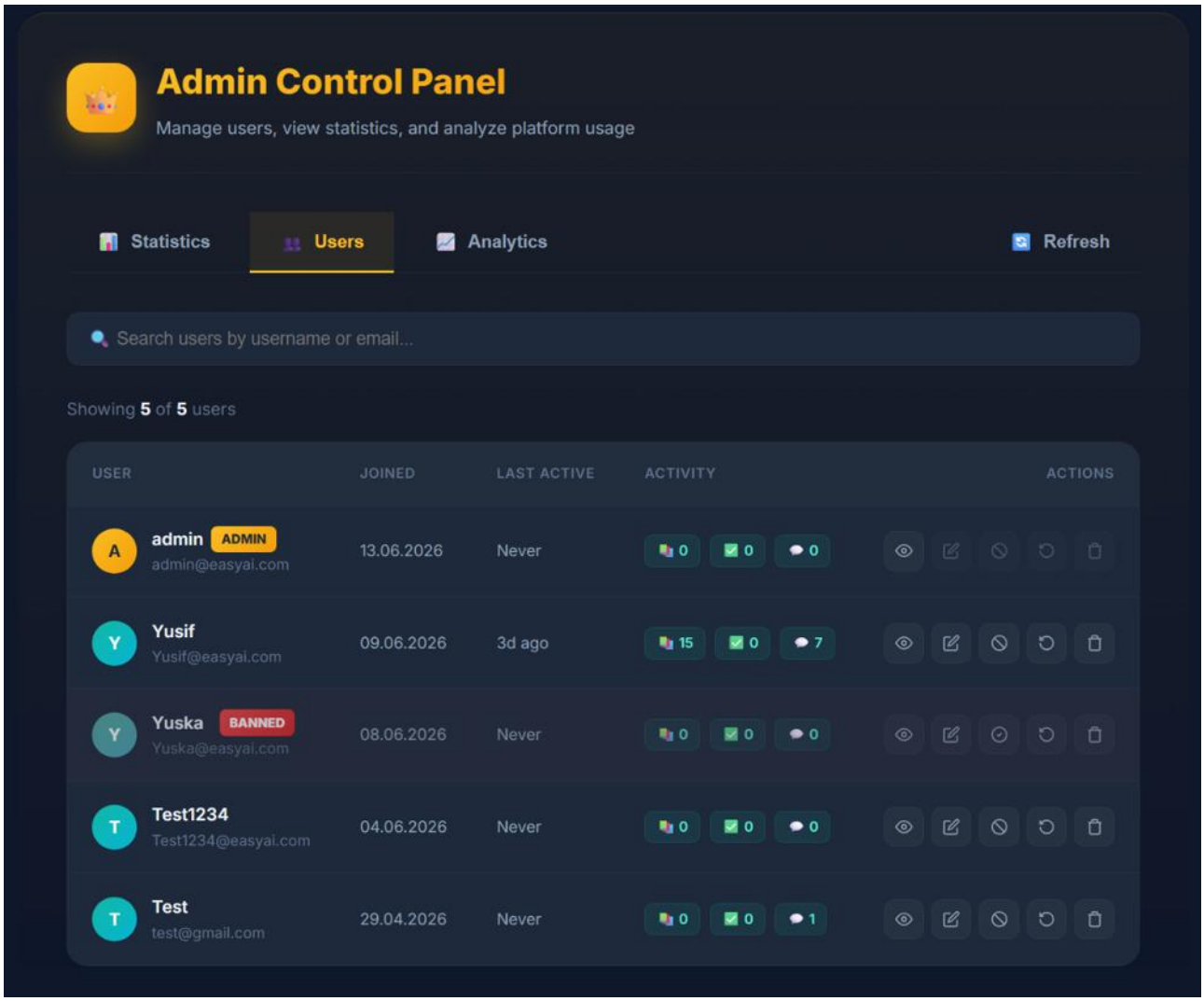
Fig. 22. Statics tab of the Admin panel on the EasyAI platform



Source: own study

The Users tab provides a detailed overview of all registered users on the platform. The admin can change the password of the registered user, view account details, reset progress, and ban the user when necessary. The administrator account itself is marked with a special admin badge to distinguish it from other users. This section can be considered as user management to edit a user's email and password since the username is a common entity to define each user (see on Fig. 23.):

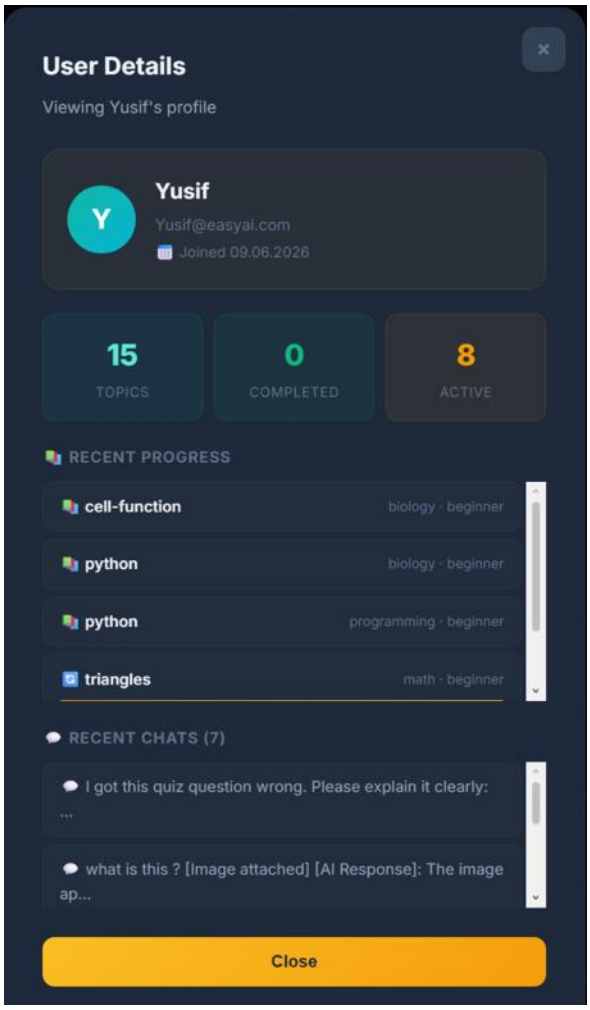
Fig. 23. Users section of Admin Control Panel



Source: own study

The view details button opens a detailed profile page with the user's activity and chat history. It serves as a separate overview of each user (see on Fig. 24.):

Fig. 24. User details tab on Admin Control Panel



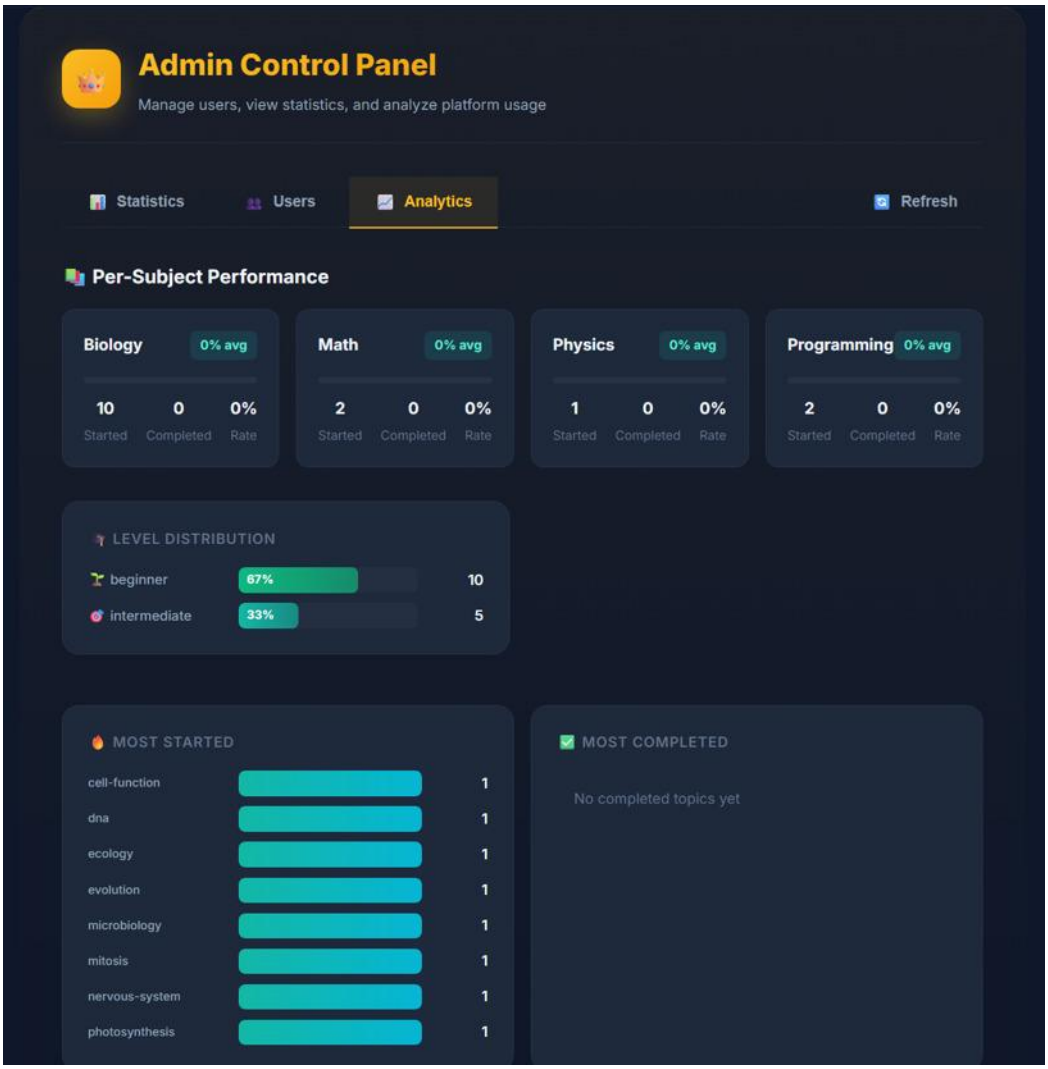
Source: own study

The edit button allows the administrator to change the user's email address or password. The reset progress button clears the user's learning history, allowing the user to start all topics from scratch. The ban and delete options allow the administrator to remove accounts from the system completely or freeze accounts in cases of spam or abuse. There is an important difference between the ban and delete actions. When the user aims to reduce the efficiency of the system by spamming AI chat, the administrator should ban accounts after giving a warning to the user. If the user continues to spam or abuse after being unbanned, the administrator can permanently delete the account. Together, these two options give the administrator a flexible way to handle account-related problems directly from the panel.

After the Statistics and the Users tabs described above, the Analytics section is the third and final section of the Admin Control Panel. While the Statistics tab presents a general overview of platform activity and the Users tab focuses on individual account management, the Analytics tab offers a

deeper view of how the platform is being used. By this section, the admin understands which subjects attract the most attention, and which topics are most started. For example, the first section shows per-subject performance through expendable cars that display started and completed topics for each topic. Below, the Level Distribution panel displays the percentage of each learning level of each topic. So, the Most Started and Most Completed bar charts complete the analytical view by showing which topics attract the user (Fig. 25.):

Fig. 25. Analytics tab of Admin Control Panel



Source: own study

The three sections of the Admin Control Panel – Statistics, Users, and Analytics work together to provide a complete administrative environment for the EasyAI platform. Through these tabs, the administrator can monitor real-time activity and manage user accounts. At the same time, the visual design of the admin panel keeps these powerful tools easy to use and understand.

Summary

This thesis presented the design and implementation of EasyAI, an AI-Powered educational platform that combines modern web technologies with locally running large language models. The goal of the project was to build a functional learning environment to deliver lessons with three defined levels of selection, adaptive and responsive quizzes, and accessible AI assistance. The development process is covered in several chapters. On the frontend side of the platform, the system was built using HTML, CSS, and JavaScript; it is organized as a single-page application that communicates through HTTP requests. The interface was designed to remain simple by using CSS. On the backend side, the platform follows a clean three-layered architecture, implemented in Python with FastAPI and SQLAlchemy ORM over a SQLite database. The API layer is divided into individual router modules – auth.py, ai.py, programming.py, progress.py, and admin.py – each module is responsible for a specific area. The service layer contains the core business logic, including the hybrid AI routing mechanism, while the data access layer manages all interactions with the database through SQLAlchemy models such as User, ChatSession, ChatMessage, and Progress. A distinctive feature of the platform is the hybrid AI integration between Groq Cloud API and local Ollama models. This mechanism routes each request to the most suitable AI model based on the topic, complexity, and required response speed. Lesson generations are handled by an Ollama model for high-quality output, while chatbot and quiz generations are implemented by Groq for fast responses. Equally important is the audience-adaptive learning approach, which allows users to choose between Beginner, Intermediate, and Expert levels of any topic. Each level uses a different teaching strategy rather than a simple difficulty modifier.

Together, these features form a complete and functional learning platform that demonstrates how modern AI technologies can be combined with web applications to build a platform at any audience level.

Bibliography

[1] Ackermann P., *Full Stack Web Development: The Comprehensive Guide*, Rheinwerk Publishing, Boston 2023

[2] Hart-Davis G., *Teach Yourself VISUALLY HTML and CSS*, John Wiley & Sons, Hoboken 2023.

[3] Wolf J., *HTML and CSS: The Comprehensive Guide*, Rheinwerk Publishing, Bonn 2023.

[4] Matthes. E., *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*, 3rd Edition, No Starch Press, San Francisco 2023.

[5] Lubanovic B., *Introducing Python: Modern Computing in Simple Packages*, O'Reilly Media, 2nd Edition, Sebastopol 2020.

[6] Brown E., *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd Edition, O'Reilly Media, Sebastopol 2020.

[7] Casciaro M., Mammino L., *Node.js Design Patterns: Design and Implement production-grade Node.js applications using proven patterns and techniques*, 3rd Edition, Packt Publishing, Birmingham 2020.

[8] Lubanovic B., *FastAPI: Modern Python Web Development*, O'Reilly Media, Sebastopol 2023.

[9] Tragura S. J. C., *Building Python Microservices with FastAPI: Build secure, scalable, and structured Python microservices from design concepts to infrastructure*, Packt Publishing, Birmingham 2022.

[10] Foster D., *Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play*, 2nd Edition, O'Reilly Media, Sebastopol, 2023.

[11] Washizaki H. (Ed.), *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) v4.0a*, IEEE Computer Society, Los Alamitos 2025.

[12] Visual Paradigm, Use Case Diagram Tutorial, https://online.visual-paradigm.com/diagrams/tutorials/use-case-diagram-tutorial/, 28.06.2026.

[13] Visual Paradigm, UML Class Diagram Tutorial, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/, 28.06.2026.

List of Figures

Fig. 1. Course page displaying study units 5
Source: https://www.khanacademy.org/math/algebra-basics/basic-alg-foundations, dated 10.05.2026 5
Fig. 2. Python task with online compiler on FreeCodeCamp web application 6
Source: https://www.freecodecamp.org/learn/python-v9/workshop-report-card-printer/step-1, dated 11.05.2026 6
Fig.3. Diagram of Client-server architecture 10
Source: own study 10
Fig. 4. Diagram describes the current HTTP Request-Response Cycle 11
Source: own study 11
Fig. 5. Developing core visualization of EasyAI 12
Source: own study 12
Fig. 6. Result of CSS compared to the previous visualization 13
Source: own study 13
Fig. 7. JavaScript change on the EasyAI platform 14
Source: own study 14
Fig. 8. Endpoints on Swagger UI 17
Source: own study 17
Fig. 9. Ollama models in a web browser 18
Source: https://ollama.com/search?q=llama, dated 20.05.2026 18
Fig. 10. EasyAI database access flow 19
Source: own study 19
Fig. 11. Use case diagram of the EasyAI platform 23
Source: own study 23
Fig.12. Class diagram of the EasyAI platform 24
Source: own study 24
Fig. 13. The whole mechanism of lesson generation on Developer PowerShell 26
Source: own study 26
Fig.14. Lesson generation and chatbot response with Groq and Ollama 27
Source: own study 27
Fig. 15. “Progress” model in the codebase 28
Source: own study 28
Fig. 16. The landing page of the EasyAI platform 29
Source: own study 29
Fig. 17. Authentication page of the EasyAI platform 30
Source: own study 30
Fig. 18. Selecting the learning level on the EasyAI platform 31
Source: own study 31
Fig. 19. Lesson interface on the EasyAI platform 32

Source: own study 32
Fig. 20. AI chatbot of the EasyAI platform 33
Source: own study 33
Fig. 21. Quiz Interface on the EasyAI platform 34
Source: own study 34
Fig. 22. Statics tab of the Admin panel on the EasyAI platform 36
Source: own study 36
Fig. 23. Users section of Admin Control Panel 37
Source: own study 37
Fig. 24. User details tab on Admin Control Panel 38
Source: own study 38
Fig. 25. Analytics tab of Admin Control Panel 39
Source: own study 39

The University of Information Technology and Management in Rzeszów

Faculty of Applied Information Technology

Diploma thesis summary

EasyAI: a Hybrid AI-Powered Educational Platform

Author: Yusif Pashazade

Supervisor: Dr inż. Łukasz Piątek

Keywords: Artificial Intelligence, E-Learning, Adaptive Learning, Large Language Models

This thesis presents the design and implementation of EasyAI, a hybrid AI-powered educational platform that combines modern web technologies with local LLM's models. The system delivers accessible lessons with lesson-level adjustment within a chatbot assistant in a single web interface. Backend is powered by hybrid AI integration between Groq cloud API and local Ollama models, which manages each request by selecting the proper AI model to send a response in JSON format. The backend is built in Python, using FastAPI and SQLAlchemy over a SQL database, while the frontend uses HTML, CSS and vanilla JavaScript. Another key feature is the audience-adaptive learning approach, which lets users choose between Beginner, Intermediate, and Expert levels for any topic, each using a different teaching strategy. Together, these components complete an educational platform that demonstrates how modern AI models can improve efficiency and performance in education.